

RICA Multi-Agent Feature — Technical Documentation

Document Version: 4.0
Date: 2026-04-16
Status: Implementation Complete — Batch Spawn, Orchestrator Patterns, Conflict Detection & Token Management Analysis
Base Branch: `main` (commit `7bba82c23`)

Table of Contents

- 1. [Executive Summary](#)
- 2. [Baseline: RICA Before Multi-Agent](#)
- 3. [Architecture Overview](#)
- 4. [Implementation Details](#)
 - 4.1 [Core Backend — New Files](#)
 - 4.2 [Core Backend — Modified Files](#)
 - 4.3 [GUI Frontend — New Files](#)
 - 4.4 [GUI Frontend — Modified Files](#)
- 5. [Data Flow & Sequence Diagrams](#)
- 6. [Type System & Interfaces](#)
- 7. [Orchestrator Intelligence](#)
- 8. [Agent Roles & Specialization](#)
- 9. [Parallel Execution Engine](#)
- 10. [Progress Tracking & Real-Time UI](#)
- 11. [Tool Argument Summarization](#)
- 12. [Abort & Cancellation Architecture](#)
- 13. [File Change Summary](#)
- 14. [v2.0 — Session Lifecycle, Kill Controls & UI Hardening](#)
- 15. [Bugs Encountered & Resolutions](#)
- 16. [Known Limitations & Future Work](#)
- 17. [v3.0 — Token Usage Tracking, API Usage Capture & Session Token UI](#)
- 18. [v4.0 — Batch Spawn, Orchestrator Patterns, Conflict Detection & Token Management](#)
 - 18.1 [Overview](#)
 - 18.2 [Batch Agent Spawning \(builtin_spawn_agents\)](#)
 - 18.3 [Orchestrator Pattern System](#)
 - 18.4 [Advanced Agent Infrastructure](#)
 - 18.5 [Task Analysis Tool \(builtin_analyze_task\)](#)
 - 18.6 [Pre-flight File Overlap Detection](#)
 - 18.7 [Post-flight Conflict Detection \(FileTracker\)](#)
 - 18.8 [BatchAgentsDiv — Animated Robot UI](#)

- 18.9 [Token Management Analysis & Recommendations](#)
- 18.10 [New Files \(v4.0\)](#)
- 18.11 [Modified Files \(v4.0\)](#)
- 18.12 [Updated Limitations & Future Work](#)

1. Executive Summary

RICA (formerly a fork of Continue, the open-source AI coding assistant) was extended with a **multi-agent orchestration system** that enables an LLM orchestrator to decompose complex tasks into subtasks and delegate them to specialized, autonomous sub-agents running in parallel or sequentially.

Before: RICA operated in two modes — `chat` (conversational) and `agent` (single-agent with tool access). All tool execution was sequential and single-threaded.

After: RICA now supports a third mode — `multi-agent` — where an orchestrator LLM intelligently decomposes tasks and spawns specialized sub-agents (Researcher, Implementer, Tester, Reviewer, Analyst, Planner). Each sub-agent runs autonomously with its own conversation context and full tool access. Multiple agents can execute in parallel via `Promise.all`. A persistent UI panel displays real-time agent status, tool usage with argument details, thinking logs, and results.

Key Metrics

Metric	v1.0	v2.0	v3.0	v4.0
New files created	10	11 (+SubAgentRegistry)	12 (+selectSessionTokens)	22 (+10 batch/patterns/conflict)
Existing files modified	22	30	34	40
Total lines added	~2,500+ (new) + ~600 (mods)	~3,200+ (new) + ~1,100 (mods)	~3,400+ (new) + ~1,500 (mods)	~5,900+ (new) + ~1,700 (mods)
New TypeScript interfaces	5	5 (+sessionId fields)	6 (+LLMCallDetail)	
New protocol messages	1 (<code>subAgentProgress</code>)	3 (+ <code>cancelSubAgent</code> , <code>killSession</code>)	3	
New Redux selectors	0	0	1 (<code>selectSessionTokens</code>)	
New built-in tool	1 (<code>builtin_spawn_agent</code>)	1	1	
Agent roles defined	6	6	6	

Metric	v1.0	v2.0	v3.0	v4.0
Max tool iterations per agent	25	25	25	
Token tracking	None	None	Full (API + tiktoken fallback)	

2. Baseline: RICA Before Multi-Agent

Prior to the multi-agent feature, RICA's architecture was:

2.1 Mode System

```
MessageModes = "chat" | "agent"
```

- **Chat mode:** Conversational responses only, no tool access
- **Agent mode:** Single LLM agent with access to built-in tools (read files, edit files, grep, glob, terminal commands, etc.)

2.2 Tool Execution

- All tools defined in `core/tools/builtIn.ts` — 12 built-in tools:
 - `builtin_read_file` , `builtin_edit_existing_file` , `builtin_create_new_file`
 - `builtin_run_terminal_command` , `builtin_grep_search` , `builtin_file_glob_search`
 - `builtin_ls` , `builtin_search_web` , `builtin_view_diff`
 - `builtin_read_currently_open_file` , `builtin_create_rule_block` , `builtin_request_rule`
- Tool execution was **strictly sequential** — one tool call at a time
- Tool calls were dispatched via `gui/src/redux/thunks/callToolById.ts` → Core `tools/call` handler → `callTool()` → tool implementation
- No mechanism for parallel tool execution

2.3 Conversation Flow

```
User Input → streamNormalInput → LLM response → (if tool calls) → callToolById → streamResponseAfterToolCall → LLM continues
```

Single-threaded, single-agent loop. The LLM could use tools iteratively but had one conversation context and one identity.

2.4 UI

- `gui/src/pages/gui/Chat.tsx` rendered chat history as a flat list
- Tool calls rendered inline via `ToolCallDiv` components

- No concept of sub-agent panels, progress tracking, or orchestration visualization

2.5 Protocol

- `core/protocol/webview.ts` (`ToWebviewFromIdeOrCoreProtocol`) — messages from Core to GUI
- `core/protocol/core.ts` (`ToCoreFromIdeOrWebviewProtocol`) — messages from GUI to Core
- No sub-agent-related protocol messages existed

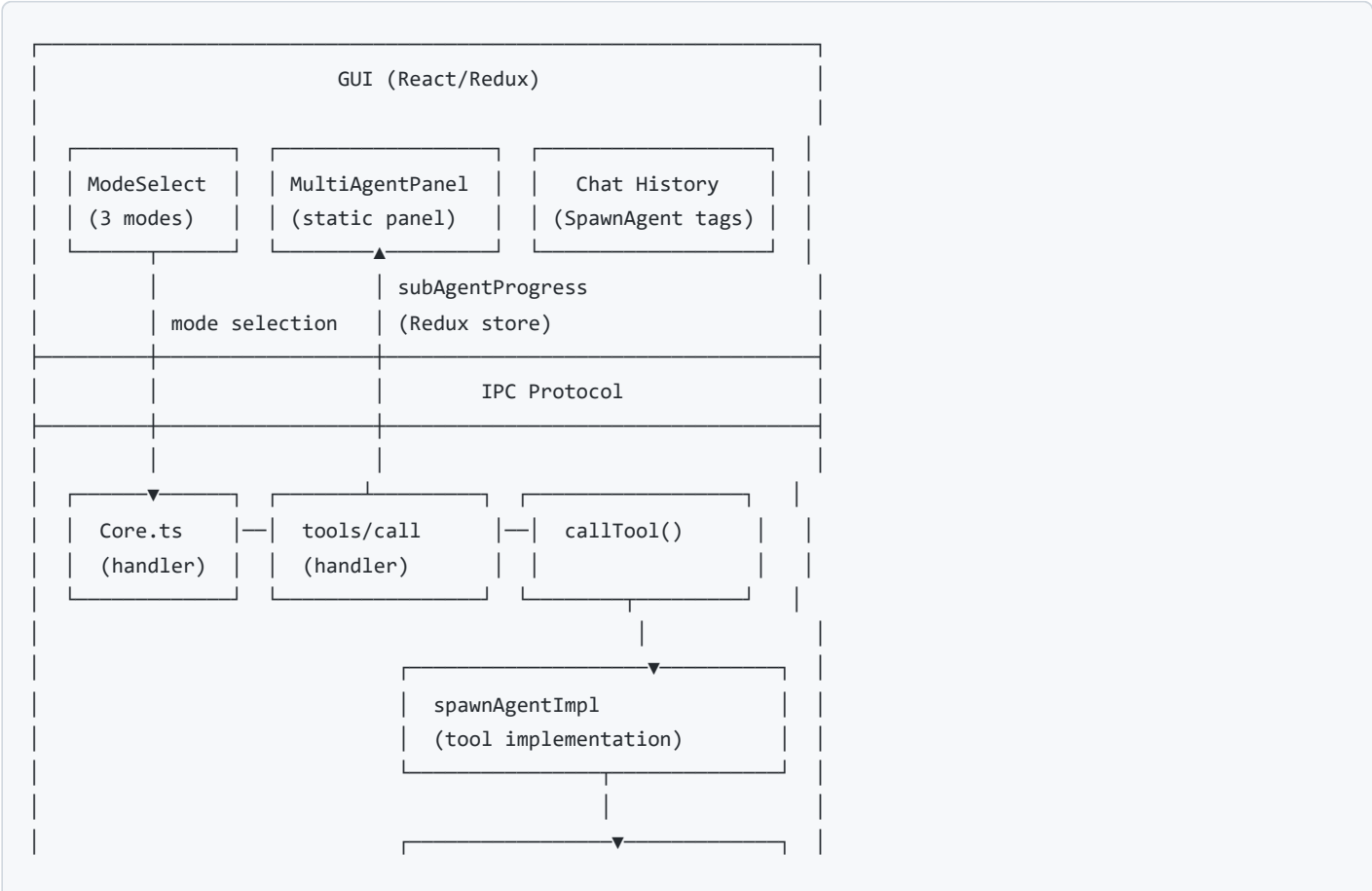
2.6 Redux State

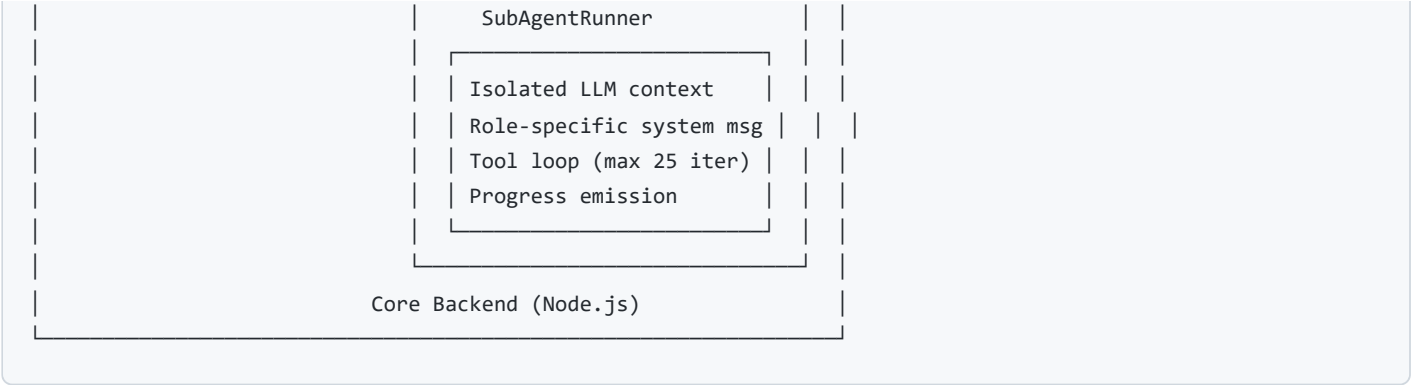
```
type SessionState = {
  history: ChatHistoryItemWithMessageId[];
  isStreaming: boolean;
  mode: MessageModes; // "chat" | "agent"
  toolCallStates: ToolCallState[];
  // ... other fields
};
```

No `subAgentProgress` field. No mechanism to track parallel agent state.

3. Architecture Overview

The multi-agent system follows a **hub-and-spoke** architecture:





Key Design Decisions

- 1. **Sub-agents cannot spawn sub-agents** — `SpawnAgent` is filtered from sub-agent tool lists to prevent unbounded recursion
- 2. **Sub-agents use the same LLM** as the orchestrator (`config.selectedModelByRole.chat`)
- 3. **EditExistingFile has a special sub-agent implementation** — writes directly via IDE API instead of going through the GUI diff UX
- 4. **Progress is streamed via protocol messages** — not through the conversation history
- 5. **Parallel execution uses `Promise.all`** — auto-approved tool calls fire concurrently
- 6. **v2.0: SubAgentRegistry is the single source of truth** for active agent state, enabling per-agent and session-level cancellation without relying on frontend state
- 7. **v2.0: Session-tagged agents** — every agent is tagged with its originating session ID for isolation and batch cleanup
- 8. **v2.0: Agent progress persists to disk** — `subAgentProgress` is saved with the session and restored on revisit

4. Implementation Details

4.1 Core Backend — New Files

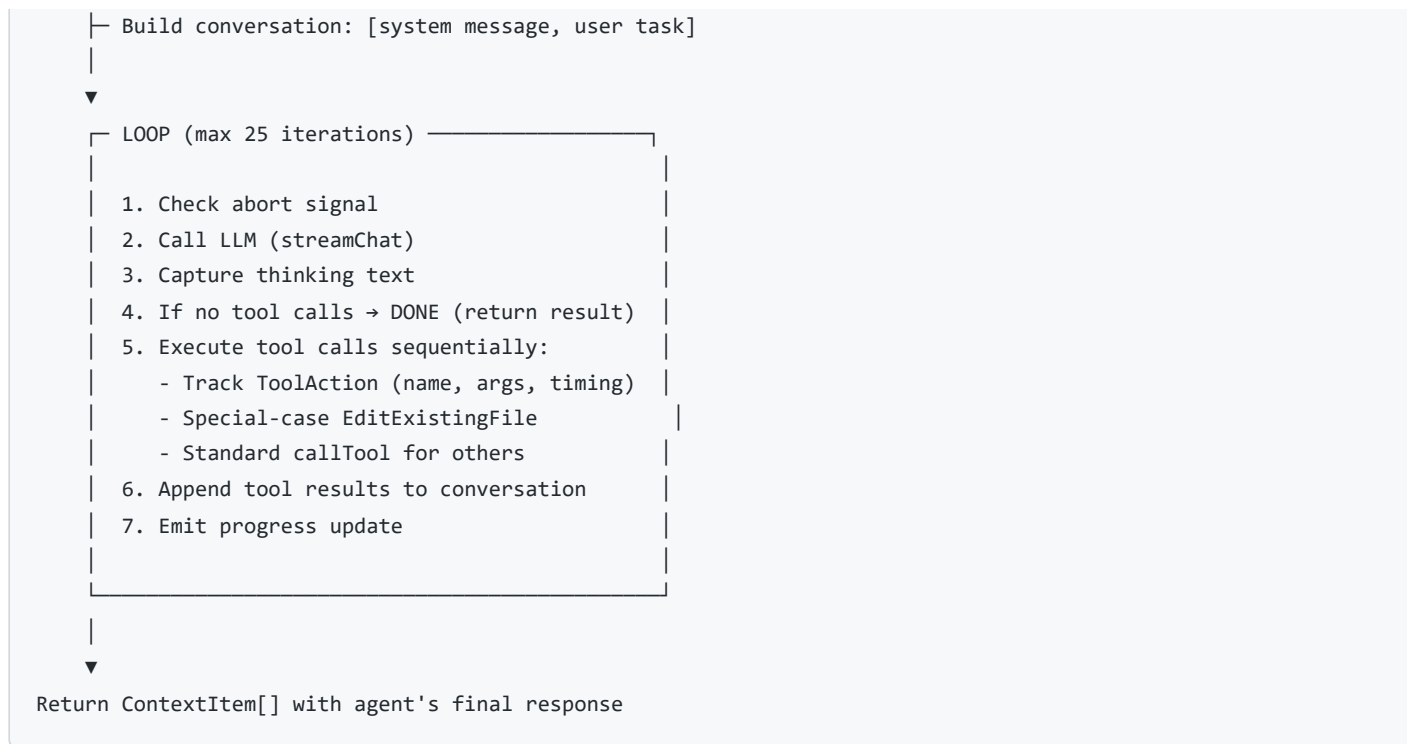
`core/tools/multiagent/SubAgentRunner.ts` (578 lines)

The execution engine for sub-agents. Each sub-agent gets:

- Its own `ChatMessage[]` conversation (starting with role-specific system message + task)
- An `AbortController` linked to the parent's abort signal
- Access to all built-in tools except `SpawnAgent`
- A maximum of 25 tool-call iterations before forced completion

Lifecycle:

```
constructor(ide, config, fetchFn, parentAbortSignal, sendProgress)
|
▼
run(subAgentId, task, description, role, tools, model)
|
├ Create child AbortController (linked to parent)
├ Filter tools (remove SpawnAgent)
```



Key methods:

- `run()` — Main execution loop
- `callLLM()` — Streams LLM response, assembles full content + tool call deltas
- `mergeToolCallDeltas()` — Handles incremental tool call streaming
- `extractToolCalls()` — Validates and normalizes tool calls from LLM response
- `summarizeToolArgs()` — Extracts human-readable summaries from tool arguments (e.g., file paths, search queries)

`core/tools/definitions/spawnAgent.ts` (51 lines)`

JSON Schema tool definition registered as builtin_spawn_agent:

```
{
  name: "builtin_spawn_agent",
  parameters: {
    task: string,          // Detailed instructions for the sub-agent
    description: string,   // One-line UI description
    role?: enum            // "Researcher" | "Implementer" | "Tester" | "Reviewer" | "Analyst" | "Planner"
  }
}
```

UI metadata for rendering in chat:

- `wouldLikeTo` : "delegate a task to a sub-agent: {description}"
- `isCurrently` : "running sub-agent: {description}"
- `hasAlready` : "completed sub-agent task: {description}"

`core/tools/implementations/spawnAgent.ts` (57 lines)

Bridge between the tool call system and SubAgentRunner :

1. Validates required `task` and `description` parameters
2. Gathers available tools from config (filters out `SpawnAgent`)
3. Generates a UUID for the sub-agent
4. Creates `SubAgentRunner` with IDE, config, fetch, abort signal, and progress callback
5. Calls `runner.run()` and returns the result as `ContextItem[]`

`core/tools/implementations/editFileForSubAgent.ts` (89 lines)

Specialized file editor for sub-agents that bypasses the GUI diff workflow:

- Resolves relative paths against IDE workspace directories
- Detects "lazy placeholders" (e.g., `// ... existing code`, `# ... existing`) via regex
- If placeholders found: uses LLM to merge the agent's proposed changes with existing file content
- If no placeholders: writes content directly via `ide.writeFile()`
- Strips markdown code blocks that LLMs sometimes wrap around code

`core/llm/multiAgentSystemMessage.ts` (63 lines)

The system prompt that transforms the LLM into an orchestrator. Defines:

- **Decision framework** — when to reply directly vs. spawn agents
- **Execution patterns** — parallel (independent tasks), sequential (dependent phases), single agent
- **Anti-duplication rules** — no overlapping scopes, split by area not function
- **Role definitions** — 6 specialized roles with descriptions
- **Task description quality** — good/bad examples for specificity

`core/tools/multiagent/SubAgentRegistry.ts` (65 lines) — ****v2.0 NEW****

Centralized singleton registry for tracking all active sub-agents:

```
class SubAgentRegistry {
  private agents = Map<string, { abortController: AbortController; sessionId?: string }>;

  register(subAgentId, abortController, sessionId?) // Called on agent start
  unregister(subAgentId) // Called on agent completion/error
  cancelAgent(subAgentId): boolean // Aborts one agent
  cancelAllForSession(sessionId): number // Batch-cancel by session
  cancelAll(): number // Nuclear option
}
```

The registry is the backend source of truth for agent lifecycle. The frontend may have stale state (e.g., after a session switch), but the registry always knows which agents are genuinely running.

`core/tools/multiagent/index.ts` (1 line)

Barrel export: `export { SubAgentRunner } from './SubAgentRunner.js';`

4.2 Core Backend — Modified Files

`core/index.d.ts` — Type Definitions

Added 5 new types/interfaces:

```
export type MessageModes = "chat" | "agent" | "multi-agent"; // was: "chat" | "agent"

export type SubAgentStatus = "running" | "completed" | "errored" | "canceled";

export interface ToolAction {
  toolName: string;
  timestamp: number;
  status: "running" | "completed" | "errored";
  durationMs?: number;
  argSummary?: string; // Human-readable summary of tool arguments
}

export interface ThinkingEntry {
  iteration: number;
  content: string;
  timestamp: number;
}

export interface SubAgentProgress {
  subAgentId: string;
  parentToolCallId: string;
  task: string;
  description: string;
  role?: string;
  status: SubAgentStatus;
  currentAction?: string;
  stepsCompleted: number;
  result?: string;
  error?: string;
  toolActions: ToolAction[];
  thinkingLog: ThinkingEntry[];
  startedAt: number;
  completedAt?: number;
}
```

Extended `ToolExtras` with sub-agent support:

```
export interface ToolExtras {
  // ... existing fields ...
  abortSignal?: AbortSignal; // NEW
  sendSubAgentProgress?: (progress: SubAgentProgress) => void; // NEW
}
```

Added `index?: number` to `ToolCallDelta` for parallel tool call matching.

`core/core.ts` — Protocol Handler

- **tools/call handler** — now accepts `messageId`, creates an `AbortController`, defines `sendSubAgentProgress` callback that forwards progress to GUI via `messenger.send("subAgentProgress", progress)`, and passes both `abortSignal` and `sendSubAgentProgress` through `callTool()`

`core/protocol/webview.ts` — Protocol Messages

Added new message type:

```
subAgentProgress: [SubAgentProgress, void];
```

`core/protocol/passThrough.ts` — Protocol Pass-Through

Added `"subAgentProgress"` to the pass-through list so VS Code extension forwards it.

`core/tools/builtin.ts` — Built-In Tool Registry

Added:

```
SpawnAgent = "builtin_spawn_agent",
```

`core/tools/callTool.ts` — Tool Dispatcher

Passes `extras.abortSignal` and `extras.sendSubAgentProgress` through to tool implementations.

`core/tools/index.ts` — Tool Registration

Imports and registers `spawnAgentTool` and `spawnAgentImpl`.

`core/llm/countTokens.ts` — Token Counting

Added handling for multi-agent mode system message injection when counting tokens.

`core/llm/messages.ts` — Message Construction

Added multi-agent system message to the conversation when mode is `"multi-agent"`.

`core/llm/openaiTypeConverters.ts` — LLM Response Parsing

Enhanced `fromChatCompletionChunk` to handle parallel tool call deltas with `index` field for correct tool call matching during streaming.

4.3 GUI Frontend — New Files

`gui/src/pages/gui/MultiAgentPanel.tsx` (843 lines)

Persistent panel rendered above the chat area. Visible when `subAgentProgress` contains entries.

Component hierarchy:

```
MultiAgentPanel
├─ PanelHeader (collapsible, shows aggregate status)
├─ PanelBody (when expanded)
│   └─ SVG connection lines (animated for running agents)
│   └─ OrchestratorNode (centered pill, clickable)
│   └─ AgentCards (flex-wrap row)
│       └─ AgentCard (compact: role icon, name, status dot, steps, current action, elapsed time)
├─ CollapsedSummary (when collapsed: inline strip)
├─ AgentDetailOverlay (modal: full task, thinking log, tool timeline, result)
└─ OrchestratorDetailOverlay (modal: aggregate stats across all agents)
```

Sub-components:

- `RoleIcon` — Maps roles to Heroicons (MagnifyingGlass, CodeBracket, Beaker, Eye, ChartBar, ClipboardDocumentList, CpuChip)
- `StatusDot` — Animated color dot (blue pulse = running, green = completed, red = errored, yellow = canceled)
- `StatusBadge` — Pill badge with icon and text
- `ToolTimeline` — Chronological list of tool executions with name, arg summary, duration, and status icon
- `ThinkingLog` — Expandable iteration-by-iteration reasoning log with markdown rendering
- `AgentCard` — Compact 144px card showing metrics only (no response text)
- `AgentDetailOverlay` — Full-screen modal with task description, thinking log, tool timeline, result/error

`gui/src/pages/gui/ToolCallDiv/MultiAgentTreeView.tsx` (500 lines)

Alternative inline tree view rendered within the chat conversation history (alongside tool calls). Provides expandable agent nodes with SVG connection lines.

`gui/src/pages/gui/ToolCallDiv/SubAgentDiv.tsx` (157 lines)

Compact inline component for a single sub-agent's tool call result in chat history. Collapsible header showing status, description, step count, and result.

4.4 GUI Frontend — Modified Files

`gui/src/pages/gui/Chat.tsx`

Major changes:

1. **MultiAgentPanel mount** — inserted between `TabBar` and `StepsDiv`
2. **subAgentProgress listener** — `useWebviewListener("subAgentProgress", ...)` dispatches to Redux
3. **SpawnAgent-aware rendering** — chat history separates SpawnAgent tool calls from regular tool calls. SpawnAgent calls render as compact inline tags (`"Spawned Researcher: ..."`) instead of full ToolCallDiv components. Regular tool calls render normally.

`gui/src/redux/slices/sessionSlice.ts`

1. **New state field:** `subAgentProgress: Record<string, SubAgentProgress>`
2. **New reducer:** `updateSubAgentProgress` — stores progress keyed by `subAgentId`
3. **Fixed streamUpdate reducer** — rewrote tool call delta merging logic to handle parallel tool calls correctly.

- Each tool result gets its own history item (role-based new item condition)
- Tool call deltas merged by ID matching (not index replacement)
- Streaming chunks that carry both content and toolCalls processed independently

`gui/src/redux/thunks/streamNormalInput.ts`

1. **Multi-agent mode recognition** — added "multi-agent" to the mode check
2. **Parallel tool call execution** — when multiple tool calls are auto-approved:

```
if (autoApproved.length > 1) {
  const promises = autoApproved.map((tc) =>
    dispatch(callToolById({ toolCallId: tc.toolCallId })).then(unwrapResult)
  );
  await Promise.all(promises);
}
```

This enables all SpawnAgent calls to fire concurrently.

3. Sequential fallback for single or non-auto-approved tool calls.

`gui/src/redux/thunks/streamResponseAfterToolCall.ts`

Updated to collect tool results for ALL parallel tool calls (not just the first) and send them back to the LLM in a single continuation request. This enables the orchestrator to see all agent results before responding.

`gui/src/redux/thunks/callToolById.ts`

Extended to pass `messageId` through the `tools/call` protocol message, enabling abort controller association on the Core side.

`gui/src/components/ModeSelect/ModeSelect.tsx`

Added "Multi-Agent" as a third mode option in the dropdown:

```
{ value: "multi-agent", label: "Multi-Agent", icon: <CpuChipIcon /> }
```

`gui/src/components/ModeSelect/ModelIcon.tsx`

Added `CpuChipIcon` rendering for the "multi-agent" mode.

`gui/src/redux/selectors/selectActiveTools.ts`

Added `SpawnAgent` to the active tools list when mode is "multi-agent".

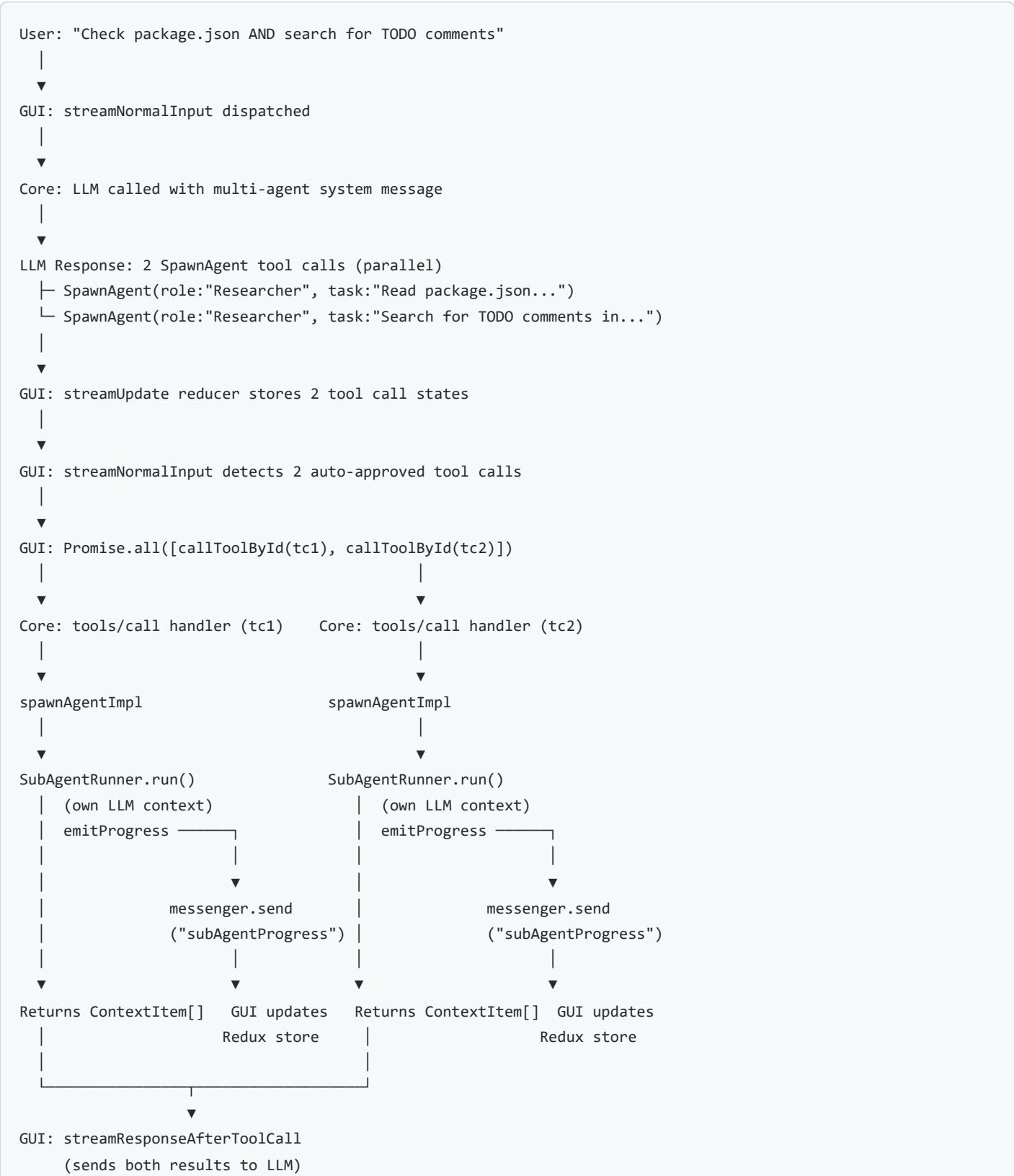
`gui/src/redux/slices/uiSlice.ts`

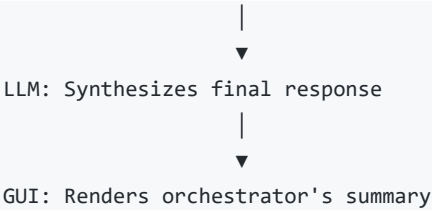
Added default tool setting for `builtin_spawn_agent` as "allowedWithoutPermission" — sub-agents auto-execute without user approval.

`gui/src/index.css`

5. Data Flow & Sequence Diagrams

5.1 Normal Flow (Parallel Agents)





5.2 Protocol Message Flow

GUI → Core:	Core → GUI:
<code>tools/call { toolCall, messageId }</code>	<code>subAgentProgress { subAgentId, status, sessionId, ... }</code>
<code>cancelSubAgent { subAgentId }</code>	<code>{ success: boolean }</code>
<code>killSession { sessionId }</code>	<code>{ killedCount: number }</code>
	<code>subAgentProgress (repeated per progress update)</code>
	<code>toolCallPartialOutput { ... }</code>

6. Type System & Interfaces

Core Types (core/index.d.ts)

```
// Mode system (extended)
export type MessageModes = "chat" | "agent" | "multi-agent";

// Sub-agent lifecycle status
export type SubAgentStatus = "running" | "completed" | "errored" | "canceled";

// Individual tool execution record within a sub-agent
export interface ToolAction {
  toolName: string;      // e.g., "builtin_grep_search"
  timestamp: number;     // Unix ms when tool execution started
  status: "running" | "completed" | "errored";
  durationMs?: number;   // Time taken for tool execution
  argSummary?: string;   // Human-readable: e.g., '"useState"' for grep, 'App.tsx' for read
}

// Sub-agent reasoning step
export interface ThinkingEntry {
  iteration: number;     // 1-indexed loop iteration
  content: string;       // LLM's text response for this iteration
  timestamp: number;     // Unix ms
}

// Full sub-agent state (streamed to GUI in real-time)
export interface SubAgentProgress {
  subAgentId: string;    // UUID
  parentToolCallId: string; // Links to orchestrator's SpawnAgent tool call
  task: string;          // Full task description given to agent
```

```

description: string;           // Short one-line description
role?: string;                 // "Researcher" | "Implementer" | etc.
status: SubAgentStatus;
currentAction?: string;        // What the agent is doing right now
stepsCompleted: number;        // Number of tool calls executed
result?: string;               // Final response text (when completed)
error?: string;                // Error message (when errored)
toolActions: ToolAction[];     // Full timeline of tool executions
thinkingLog: ThinkingEntry[]; // Iteration-by-iteration reasoning
startedAt: number;             // Unix ms
completedAt?: number;          // Unix ms (set on terminal states)
sessionId?: string;            // v2.0: originating session for isolation
}

// v2.0: Session now persists agent progress
export interface Session {
  sessionId: string;
  title: string;
  workspaceDirectory: string;
  history: ChatHistoryItem[];
  subAgentProgress?: Record<string, SubAgentProgress>; // v2.0: persisted agent tree
}

// Extended tool extras (core/index.d.ts)
export interface ToolExtras {
  ide: IDE;
  llm: ILLM;
  fetch: FetchFunction;
  tool: Tool;
  toolCallId?: string;
  onPartialOutput?: (...) => void;
  config: ContinueConfig;
  abortSignal?: AbortSignal; // v1.0
  sendSubAgentProgress?: (progress: SubAgentProgress) => void; // v1.0
  sessionId?: string;        // v2.0
}

```

Redux State (gui/src/redux/slices/sessionSlice.ts)

```

type SessionState = {
  // ... existing fields ...
  subAgentProgress: Record<string, SubAgentProgress>; // NEW: keyed by subAgentId
};

```

7. Orchestrator Intelligence

The orchestrator's behavior is defined by the system message in `core/llm/multiAgentSystemMessage.ts`. It evolved through two major iterations:

Version 1 (Initial — Aggressive Spawning)

MANDATORY: For every user request, you MUST spawn at least 2 agents with different roles.
NEVER do work yourself – you ALWAYS delegate.

Problem: Even "Hi" or "what can you do?" spawned 2 agents unnecessarily.

Version 2 (Current — Smart Dispatch)

DECISION FRAMEWORK:
A) REPLY DIRECTLY – greetings, simple questions, clarifications, trivial requests
B) SPAWN AGENTS – when task requires file access, code changes, or investigation

PHASED EXECUTION:
1) PARALLEL – independent subtasks spawn simultaneously
2) SEQUENTIAL – dependent tasks spawn in phases (research → implement → test)
3) SINGLE AGENT – focused tasks need only one specialist

PREVENTING DUPLICATE WORK:
- Never spawn two agents with same role doing overlapping tasks
- Split by AREA not FUNCTION
- Give explicit file/directory boundaries

The phased execution is supported by the existing architecture — after all agents in Phase 1 complete, `streamResponseAfterToolCall` sends their results back to the orchestrator, which can then spawn Phase 2 agents with context from Phase 1.

8. Agent Roles & Specialization

Each role gets a distinct system message in `SubAgentRunner.ts` :

Role	System Message Focus	Typical Tools Used
Researcher	Explore codebase, read files, search patterns. Do NOT modify files. Report with file paths and line numbers.	ReadFile, GrepSearch, FileGlobSearch, LS
Implementer	Write/edit code. Read before changing. Minimal edits, follow conventions. Verify changes compile.	ReadFile, EditExistingFile, CreateNewFile, RunTerminalCommand
Tester	Run test suites, validate correctness. Report pass/fail clearly. Check regressions.	RunTerminalCommand, ReadFile, GrepSearch
Reviewer	Check correctness, security, edge cases, code style. Provide specific actionable feedback.	ReadFile, GrepSearch, FileGlobSearch
Analyst	Investigate bugs, reproduce problems, trace root causes systematically.	ReadFile, GrepSearch, RunTerminalCommand, LS

Role	System Message Focus	Typical Tools Used
Planner	Create step-by-step implementation plans with specific file paths and function names.	ReadFile, GrepSearch, FileGlobSearch, LS

9. Parallel Execution Engine

How It Works

In `gui/src/redux/thunks/streamNormalInput.ts` :

```
// After LLM responds with tool calls:
const autoApproved = toolCallStates.filter(
  (tc) => toolSettings[tc.toolCall.function.name] === "allowedWithoutPermission"
);

if (autoApproved.length > 1) {
  // PARALLEL: fire all at once
  const promises = autoApproved.map((tc) =>
    dispatch(callToolById({ toolCallId: tc.toolCallId })).then(unwrapResult)
  );
  await Promise.all(promises);
} else {
  // SEQUENTIAL: one at a time
  for (const tc of autoApproved) {
    const response = await dispatch(callToolById({ toolCallId: tc.toolCallId }));
    unwrapResult(response);
  }
}
```

Why SpawnAgent Auto-Approves

In `gui/src/redux/slices/uiSlice.ts` , `builtin_spawn_agent` is set to `"allowedWithoutPermission"` by default, meaning the user is not prompted for each agent spawn.

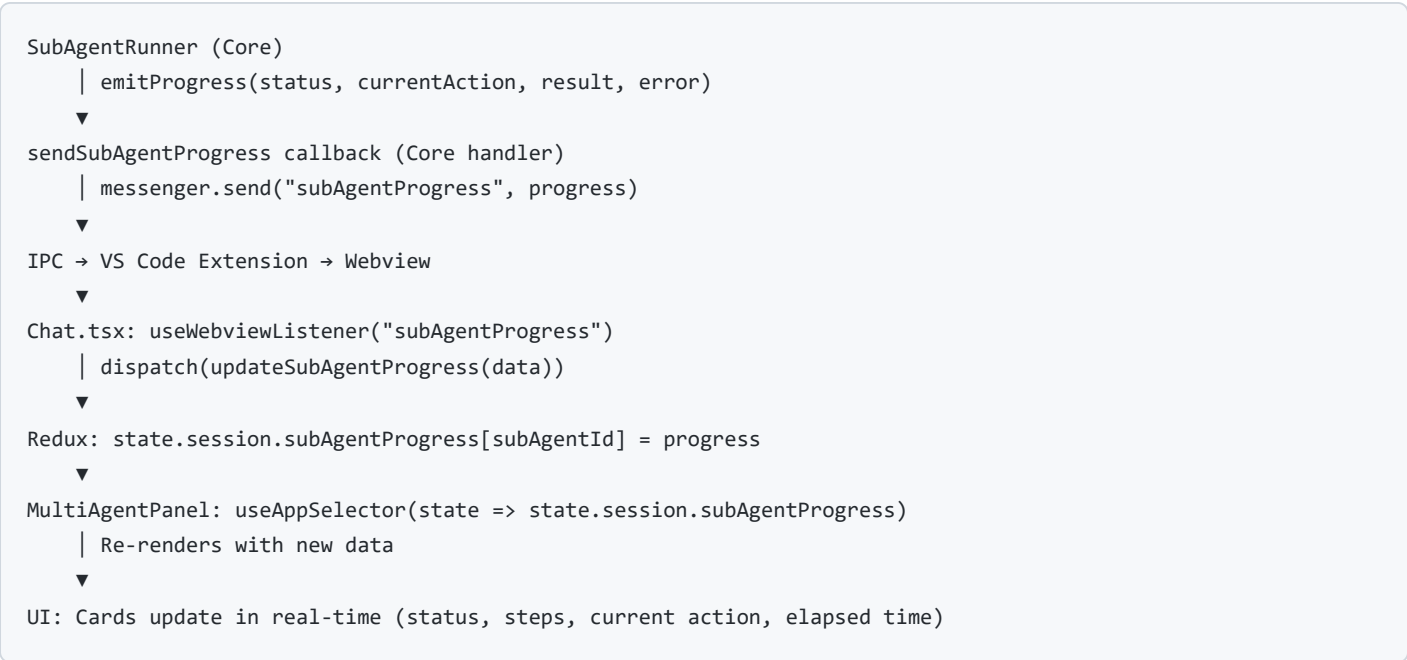
Streaming Challenges Solved

Parallel tool calls introduced a streaming challenge: the LLM sends tool call deltas incrementally, and multiple tool calls can arrive interleaved in a single stream. The `sessionSlice.ts` `streamUpdate` reducer was rewritten to:

- 1. Match deltas by `id` (not array index) to handle out-of-order arrival
- 2. Merge argument fragments into existing tool call states (not replace)
- 3. Process content and toolCalls independently (a chunk can carry both)
- 4. Create separate history items for each tool result (each has a unique `toolCallId`)

10. Progress Tracking & Real-Time UI

Data Flow



What's Tracked Per Agent

Field	Updated When	Shown Where
status	Every state change	StatusDot, StatusBadge
currentAction	Every tool execution start	AgentCard, collapsed strip
stepsCompleted	After each tool call	AgentCard metrics
toolActions[]	Each tool start/complete	ToolTimeline in detail overlay
thinkingLog[]	Each LLM iteration	ThinkingLog in detail overlay
result	On completion	AgentDetailOverlay
error	On failure	AgentDetailOverlay, AgentCard
startedAt / completedAt	Start / terminal state	Elapsed time display

11. Tool Argument Summarization

Added in the final iteration to enrich agent cards with tool context.

`summarizeToolArgs()` in `SubAgentRunner.ts`

Extracts human-readable summaries from tool JSON arguments:

Tool	Extraction Logic	Example Output
<code>builtin_read_file</code>	<code>basename(args.filepath)</code>	<code>"App.tsx"</code>
<code>builtin_grep_search</code>	<code>''' + args.query + '''</code>	<code>"useState"</code>
<code>builtin_file_glob_search</code>	<code>args.pattern</code>	<code>"/**/*.test.ts"</code>
<code>builtin_run_terminal_command</code>	<code>args.command.slice(0, 80)</code>	<code>"npm run test"</code>
<code>builtin_edit_existing_file</code>	<code>basename(args.filepath)</code>	<code>"Login.tsx"</code>
<code>builtin_create_new_file</code>	<code>basename(args.filepath)</code>	<code>"utils.ts"</code>
<code>builtin_ls</code>	<code>args.dirPath</code>	<code>"src/components/"</code>
Other/unknown	First string value, 60 chars	varies

Where It Appears

- 1. `currentAction` field — enriched to show `"builtin_grep_search: \"useState\""` instead of just `"Using tool: builtin_grep_search"`
- 2. `ToolAction.argSummary` — stored per tool execution, rendered in ToolTimeline:

✓ Grep Search – "useState"1.2s

✓ Read File – App.tsx0.3s

✓ Edit Existing File – App.tsx0.8s

12. Abort & Cancellation Architecture

v2.0: Three Independent Abort Chains

A critical architectural insight discovered during hardening: the system has **three fully isolated abort chains** that do not interfere with each other:

Chain 1: STREAM CANCEL (user clicks "Cancel" or starts new session)

GUI: `cancelStream()` → `state.streamAborter.abort()`

|

▼ passed as `cancelToken` to `streamRequest()`

IdeMessenger.streamRequest abort handler

| posts "abort" with the STREAM's `messageId`

▼

Core: `abortById(streamMessageId)` → aborts llm/streamChat controller

✗ Does NOT affect tools/call requests (different `messageId`)

Chain 2: TOOL CALL ABORT (per tools/call request)

Core: tools/call handler → addMessageAbortController(toolCallMessageId)

|

▼ passed as extras.abortSignal

SubAgentRunner.parentAbortSignal

|

linked via addEventListener("abort")

▼

SubAgentRunner.abortController (child)

✗ Only fires if someone calls abortById(toolCallMessageId)

Chain 3: PER-AGENT / SESSION CANCEL (v2.0 – via SubAgentRegistry)

GUI: post("cancelSubAgent", { subAgentId })

|

▼

Core: SubAgentRegistry.cancelAgent(subAgentId)

|

entry.abortController.abort()

▼

SubAgentRunner detects abort → emits "canceled" → returns cancel message

GUI: post("killSession", { sessionId })

|

▼

Core: SubAgentRegistry.cancelAllForSession(sessionId)

|

aborts ALL agents tagged with that session

▼

All matched runners detect abort → emit "canceled"

Why This Isolation Matters

When a user clicks "Cancel All" in the StreamingToolbar:

1. Each running agent gets `cancelSubAgent` → Chain 3 fires for each
2. Main stream gets `cancelStream()` → Chain 1 fires
3. `callToolById` checks `isStreaming` after tool returns → sees `false` → skips `streamResponseAfterToolCall`
4. Cancel messages from agents are still stored in history but the orchestrator doesn't continue

When a user starts a new session:

1. `startNewSession` dispatches `killSession` → Chain 3 batch-cancels all agents
2. Stream is aborted → Chain 1
3. Session is saved with agents marked "canceled"
4. New session loads clean

v2.0: Cancel Message Design (Spawn Loop Prevention)

The cancel message returned to the orchestrator went through three iterations:

Version	Message	Problem
v1	"Agent was cancelled. This is not an error."	Orchestrator confused — thought it should retry
v2	"You may spawn a new agent for this or a different task if needed."	Spawn cascade: cancelled agents → orchestrator respawns → cancel → respawn → 3→6→9 agents
v3 (current)	"Do NOT spawn a new agent for this same task. Do NOT retry or redo this task. The user intentionally stopped it. Use the results you already have from other completed agents and move on."	Works — orchestrator takes over directly or reports what it has

SubAgentRunner Registry Integration

```
constructor(ide, config, fetchFn, parentAbortSignal, sendProgress, sessionId?)
|
▼
run(subAgentId, task, description, role, tools, model)
|
|─ Create child AbortController (linked to parent)
|─ SubAgentRegistry.register(subAgentId, abortController, sessionId) ← v2.0
|─ Filter tools (remove SpawnAgent)
|
▼
┌ LOOP (max 25 iterations) ───────────┐
│ 1. Check abort signal                │
│ 2. Call LLM → execute tools → emit progress │
└────────────────────────────────────────┘
|
|─ finally: SubAgentRegistry.unregister(subAgentId) ← v2.0
▼
Return ContextItem[]
```

Behavior on Abort

- 1. Per-agent cancel or session kill fires → child abort controller triggers
- 2. SubAgentRunner checks `abortController.signal.aborted` before each iteration
- 3. If aborted mid-loop:
 - Missing tool results get placeholder messages
 - Agent emits "canceled" status with `completedAt` timestamp
 - Returns `ContextItem` with explicit "do not respawn" message
- 4. Cleanup: parent abort listener removed + registry unregistered in `finally` block

13. File Change Summary

New Files (12)

File	Lines	Purpose	Version
core/tools/multiagent/SubAgentRunner.ts	578	Sub-agent execution engine	v1.0 + v3.0
core/tools/multiagent/SubAgentRegistry.ts	65	Centralized agent registry (singleton)	v2.0
core/tools/multiagent/index.ts	1	Barrel export	v1.0
core/tools/definitions/spawnAgent.ts	51	SpawnAgent tool schema	v1.0
core/tools/implementations/spawnAgent.ts	57	SpawnAgent tool implementation	v1.0
core/tools/implementations/editFileForSubAgent.ts	89	Sub-agent file editor	v1.0
core/llm/multiAgentSystemMessage.ts	63	Orchestrator system prompt	v1.0
gui/src/pages/gui/MultiAgentPanel.tsx	1,170	Progress panel (expanded from 843)	v1.0 + v2.0 + v3.0
gui/src/pages/gui/ToolCallDiv/MultiAgentTreeView.tsx	500	Inline tree view	v1.0
gui/src/pages/gui/ToolCallDiv/SubAgentDiv.tsx	157	Inline agent result component	v1.0
gui/src/redux/selectors/selectSessionTokens.ts	137	Session token totals memoized selector	v3.0
Total New	~2,868		

Modified Files (34)

File	Change Summary	Version
core/index.d.ts	+ 5 types, extended ToolExtras/MessageModes/ToolCallDelta; + sessionId on SubAgentProgress/ToolExtras; + subAgentProgress on Session; + token fields on PromptLog; + tokenUsage on SubAgentProgress	v1.0 + v2.0 + v3.0
core/core.ts	tools/call handler (abort, progress); + cancelSubAgent handler; + killSession handler; sessionId capture via getCurrentSessionId	v1.0 + v2.0
core/protocol/webview.ts	+ subAgentProgress message	v1.0
core/protocol/core.ts	+ cancelSubAgent , killSession protocol messages	v2.0

File	Change Summary	Version
core/protocol/passThrough.ts	+ subAgentProgress , cancelSubAgent , killSession pass-through	v1.0 + v2.0
core/tools/builtIn.ts	+ SpawnAgent enum	v1.0
core/tools/callTool.ts	Pass through abortSignal, sendSubAgentProgress, sessionId	v1.0 + v2.0
core/tools/index.ts	Register spawnAgentTool and spawnAgentImpl	v1.0
core/util/history.ts	HistoryManager.save() includes subAgentProgress in persisted session	v2.0
core/llm/countTokens.ts	Multi-agent system message token counting	v1.0
core/llm/messages.ts	Multi-agent system message injection	v1.0
core/llm/openaiTypeConverters.ts	Parallel tool call delta handling with index field	v1.0
core/config/types.ts	Mode type update; +PromptLog token fields	v1.0 + v3.0
gui/src/pages/gui/Chat.tsx	MultiAgentPanel mount, subAgentProgress listener, sessionId filtering ; +SessionTokenBadge (compact + detail card)	v1.0 + v2.0 + v3.0
gui/src/redux/slices/sessionSlice.ts	subAgentProgress state/reducer, streamUpdate rewrite; newSession restores subAgentProgress	v1.0 + v2.0
gui/src/redux/slices/uiSlice.ts	Default auto-approve for SpawnAgent	v1.0
gui/src/redux/thunks/session.ts	+ startNewSession thunk (kill agents + save session + mark cancelled)	v2.0
gui/src/redux/thunks/streamNormalInput.ts	Multi-agent mode, parallel Promise.all execution	v1.0
gui/src/redux/thunks/streamResponseAfterToolCall.ts	Collect all parallel tool results; +empty history guard	v1.0 + v2.0
gui/src/redux/thunks/callToolById.ts	messageId for abort; +session guard; +isStreaming guard; +sibling tool call detection	v1.0 + v2.0
gui/src/redux/thunks/streamThunkWrapper.tsx	Multi-agent mode; +intentional stop detection (suppress error dialog)	v1.0 + v2.0

File	Change Summary	Version
gui/src/redux/selectors/selectActiveTools.ts	SpawnAgent in active tools for multi-agent mode	v1.0
gui/src/components/ModeSelect/ModeSelect.tsx	"Multi-Agent" mode option	v1.0
gui/src/components/ModeSelect/ModeIcon.tsx	CpuChipIcon for multi-agent	v1.0
gui/src/components/mainInput/Lump/LumpToolbar/StreamingToolbar.tsx	Kill dropdown (cancel all, cancel all agents, per-agent cancel)	v2.0
gui/src/index.css	Pulse animation for status dots	v1.0
gui/src/util/index.ts	Utility updates	v1.0
core/llm/index.ts	+ <code>_lastApiUsage</code> property; OpenAI adapter usage capture (streaming + non-streaming); <code>stream_options: { include_usage: true }</code> ; <code>PromptLog</code> return sites with API-prefer-tiktoken-fallback	v3.0
core/llm/llms/stubs/RicaProxy.ts	Rica usage capture (streaming + non-streaming); debug logging	v3.0
gui/src/redux/selectors/selectSessionTokens.ts	NEW — memoized selector for session token totals, per-call breakdown, usage source detection	v3.0
gui/src/pages/gui/MultiAgentPanel.tsx	Per-agent token display (card metrics + detail overlay with reasoning/cache/source indicator)	v3.0

14. v2.0 — Session Lifecycle, Kill Controls & UI Hardening

This section documents all features added after the v1.0 document (2026-04-14 → 2026-04-15).

14.1 SubAgentRegistry — Centralized Agent Tracking

File: `core/tools/multiagent/SubAgentRegistry.ts` (NEW)

A singleton registry that tracks every active sub-agent with its `AbortController` and `sessionId`. This is the backend source of truth — the frontend may have stale state after a session switch, but the registry always knows what's genuinely running.

API:

- `register(subAgentId, abortController, sessionId)` — called in `SubAgentRunner.run()` after creating the child abort controller
- `unregister(subAgentId)` — called in the `finally` block of `SubAgentRunner.run()`
- `cancelAgent(subAgentId): boolean` — aborts one agent, removes from registry

- `cancelAllForSession(sessionId): number` — batch-cancels all agents tagged with a session
- `cancelAll(): number` — global cancel

14.2 Protocol Messages for Cancellation

Files: `core/protocol/core.ts`, `core/protocol/passThrough.ts`, `core/core.ts`

Two new protocol messages:

```
"cancelSubAgent": [{ subAgentId: string }, { success: boolean }];
"killSession":    [{ sessionId: string }, { killedCount: number }];
```

Core handlers delegate directly to `SubAgentRegistry`. Both are added to `WEBVIEW_TO_CORE_PASS_THROUGH` for the VS Code extension to forward.

14.3 Session Isolation — Progress Event Filtering

File: `gui/src/pages/gui/Chat.tsx`

The `subAgentProgress` webview listener now compares `data.sessionId` against the current session ID. Progress events from a different session are silently dropped. This prevents agents from a previous session bleeding into the current session's graph tree.

14.4 Session-Level Kill on Switch

File: `gui/src/redux/thunks/session.ts`

The `startNewSession` thunk now performs a full cleanup sequence:

1. `dispatch(setInactive())` — mark stream as stopped
2. `dispatch(abortStream())` — abort the LLM stream
3. `dispatch(clearLastEmptyResponse())` — clean up partial messages
4. `post("killSession", { sessionId })` — kill ALL backend agents for old session
5. Save old session to disk:
 - Mark "running" agents as "canceled" with `completedAt` timestamp
 - Include `subAgentProgress` in the Session object
 - Fire-and-forget `request("history/save", sessionToSave)`
6. `dispatch(newSession(session))` — switch to new/loaded session

This fires on every session switch: new session, load session, load last session.

14.5 Session Persistence — Agent Progress Survives Revisits

Files: `core/index.d.ts`, `core/util/history.ts`, `gui/src/redux/slices/sessionSlice.ts`, `gui/src/redux/thunks/session.ts`

Changes to make agent tree persist:

1. **Session interface** — added `subAgentProgress?: Record<string, SubAgentProgress>`
2. **HistoryManager.save()** — the explicit key-ordered session reconstruction now includes `subAgentProgress` (was previously stripped because it wasn't in the ordered object)

3. **newSession reducer** — restores `payload.subAgentProgress ?? {}` when loading a session
4. **saveCurrentSession** — marks "running" agents as "canceled" before persisting (agents won't be running when revisited)

14.6 Per-Agent Kill Buttons

File: `gui/src/pages/gui/MultiAgentPanel.tsx`

Running agent cards now display a kill button (StopCircleIcon). Clicking calls `ideMessenger.post("cancelSubAgent", { subAgentId })`. The button uses `e.stopPropagation()` to prevent card click from firing. Kill buttons also appear in the `AgentDetailOverlay` header with a "Cancel" label. Only visible when `agent.status === "running"`.

14.7 Kill Options Dropdown (Claude Code-Style)

File: `gui/src/components/mainInput/Lump/LumpToolbar/StreamingToolbar.tsx`

When agents are running, the streaming toolbar shows a split button:

- **Main button:** "Cancel All" (keyboard shortcut) — cancels all agents + aborts stream
- **Dropdown chevron** revealing:
 - "Cancel everything" (red XMarkIcon) — abort stream + kill all agents
 - "Cancel all agents (N)" (yellow StopCircleIcon) — kill agents, keep orchestrator stream
 - Divider
 - Per-agent entries (CpuChipIcon) — "Cancel: [Agent Role/Name]" for each running agent
- Click-outside handler closes the dropdown

When no agents are running, shows the simple "Cancel" button (unchanged from v1.0).

14.8 Expandable Agent Cards

File: `gui/src/pages/gui/MultiAgentPanel.tsx`

Agent cards can be expanded inline via a chevron toggle. Expanded cards show:

- Task description (bordered box)
- ToolTimeline (reused from detail overlay)
- ThinkingLog (last entries, expandable)
- Result/Error section

The SVG connection lines recalculate on expand/collapse via `expandedCards.size` in the dependency array.

14.9 Full-Screen Overlay Mode

File: `gui/src/pages/gui/MultiAgentPanel.tsx`

A full-screen button (ArrowsPointingOutIcon) opens the entire graph tree in a fixed overlay covering the viewport. Exit via ArrowsPointingInIcon button. Useful when many agents are running and the inline panel is too small.

14.10 Live/History Agent Split

File: `gui/src/pages/gui/MultiAgentPanel.tsx`

The expanded tree body is split into two sections:

1. **Live agents** (top, scrollable `max-h-[350px]`) — only `status === "running"` agents, with SVG connection lines from the orchestrator node
2. **Agent History** (bottom, collapsible, scrollable `max-h-[200px]`) — completed, errored, and canceled agents with `StatusBadge` tags

The **collapsed** summary strip shows only live agents. When no agents are running: "No agents running — expand to see history".

14.11 Session Guards in Tool Call Thunks

File: `gui/src/redux/thunks/callToolById.ts`

Three safety guards prevent orphaned tool results from corrupting state:

1. **Session guard** — captures `sessionIdAtStart`; after tool returns, checks if session changed → skips continuation
2. **Streaming guard** — checks `isStreaming` → if false, stream was intentionally stopped → skips continuation
3. **Sibling guard** — checks if other parallel tool calls are still running → only the last one to finish triggers `streamResponseAfterToolCall`

14.12 Intentional Stop Detection

File: `gui/src/redux/thunks/streamThunkWrapper.tsx`

The catch block previously showed an error dialog for ALL exceptions, including intentional aborts. Now checks `!getState().session.isStreaming` BEFORE dispatching `cancelStream()`. If already stopped, it was intentional (user started new session, cancelled) — suppress the error dialog.

15. Bugs Encountered & Resolutions

15.1 Spawn Cascade Loop (3 → 6 → 9 agents)

Symptom: User cancels 3 agents. Orchestrator respawns 3 new agents for the cancelled tasks. User cancels again. Orchestrator spawns 3 more. Agent count grows: 3 → 6 → 9 → ...

Root cause: The cancel message in `SubAgentRunner.ts` said "You may spawn a new agent for this or a different task if needed." The orchestrator interpreted this as instruction to retry.

Fix: Changed cancel message to explicitly prohibit respawning: "Do NOT spawn a new agent for this same task. Do NOT retry or redo this task."

Lesson: LLM-facing messages must be unambiguous. A permissive cancel message creates an infinite spawn loop because the orchestrator treats "may spawn" as "should spawn."

15.2 Error Dialog on New Session ("no user/tool message found")

Symptom: Starting a new session while agents are running produced an error dialog: "Error parsing chat history: no user/tool message found" from `compileChatMessages` in `core/llm/countTokens.ts`.

Root cause chain:

1. `startNewSession` dispatched `newSession()` which cleared history to `[]`
2. In-flight `streamResponseAfterToolCall` (from a tool call that completed during the switch) tried to compile the now-empty history
3. `compileChatMessages` expects at least one user/tool message → threw

Fix (multi-layered):

1. `startNewSession` now dispatches `setInactive()` + `abortStream()` BEFORE `newSession()` — in-flight thinks see `isStreaming=false` and bail
2. `callToolById` checks session ID and streaming state before continuing
3. `streamResponseAfterToolCall` has empty-history guard: `if (initialHistory.length === 0) return`
4. `streamThunkWrapper` detects intentional stops and suppresses error dialog

15.3 Session Persistence Not Working

Symptom: Agent tree disappeared when revisiting old sessions. Clicking through history showed no agents.

Root cause: `core/util/history.ts` `HistoryManager.save()` explicitly reconstructed the session object with only 4 fields (`sessionId`, `title`, `workspaceDirectory`, `history`), stripping `subAgentProgress`.

Fix: Added `subAgentProgress` to the ordered session reconstruction in `HistoryManager.save()`.

Lesson: When backend serialization explicitly lists fields (instead of spreading), new fields are silently dropped. Always check the serialization layer.

15.4 GUI Build Not Deploying

Symptom: All GUI changes appeared non-functional. "None of the changes worked."

Root cause: Build copy command `cp -r dist ../extensions/vscode/gui` created a nested `gui/dist/` directory instead of placing files at `gui/assets/`, `gui/index.html`, etc.

Fix: Changed to `cp -r gui/dist/* extensions/vscode/gui/` (copy contents, not the directory).

Lesson: Always verify the deployment path after build. A single wrong `cp` flag means zero UI changes are visible despite correct source code.

15.5 Agents Still Running After Session Switch

Symptom: Switching sessions or clicking "New Session" left backend agents running, consuming tokens. Returning to old sessions showed agents as "running" when they should have been killed.

Root cause: `startNewSession` only checked frontend state (`state.session.subAgentProgress`) to decide whether to kill agents. But the frontend might have already cleared state, and it never saved the session before switching.

Fix:

1. `startNewSession` now unconditionally sends `killSession` (backend registry is source of truth)
2. Before switching, saves the old session with agents marked as "canceled"
3. `saveCurrentSession` also marks running agents as "canceled" before persisting

15.6 Cancelled Agent Detection (1 of 3)

Symptom: User cancels all 3 agents, but orchestrator only detects 1 as cancelled and respawns for the other 2.

Root cause: Two agents completed before the cancel signal arrived (fast-completing tasks like file reads). The orchestrator saw 2 completed results + 1 cancel message.

Status: Partially mitigated by the stronger cancel message. The timing issue (fast agents completing before cancel arrives) is inherent to async execution. The orchestrator now uses completed results and doesn't retry cancelled tasks.

16. Known Limitations & Future Work

Current Limitations (v2.0)

1. **No per-agent cancellation from GUI** — **RESOLVED in v2.0:** kill buttons, dropdown menu, protocol messages all implemented.
2. **No token usage tracking** — **RESOLVED in v3.0:** full token tracking with real API usage capture, per-agent counts, session totals, and interactive detail card UI.
3. **No progress percentage** — `stepsCompleted` is a count, not a percentage. There's no way to estimate total steps in advance.
4. **No agent-to-agent direct communication** — agents are fully isolated. Coordination happens only through the orchestrator (sequential phases). **Design decision confirmed in v4.0:** isolation model is correct. ContextBridge provides orchestrator-mediated context sharing between sequential phases. See §18.4.
5. **No retry on failure** — **RESOLVED in v4.0:** FaultHandler provides automatic retry with progressive backoff for transient errors (rate limits, timeouts). Non-retryable errors are escalated. See §18.4.
6. **Sub-agents share the same LLM model** — no ability to assign different models to different agent roles.
7. **Debug logging present** — `[PARALLEL-DEBUG]`, `[PARALLEL-DEBUG-CORE]`, `[SubAgent:...]` console.log statements remain. Should be removed or gated behind a debug flag before release.
8. **Cancel timing race** — fast-completing agents may finish before a cancel signal arrives. The frontend shows them as "completed" even if the user intended to cancel everything.

Future Work

- **Token usage tracking** — **DONE in v3.0**
- **Collaborative workflows** — **DONE in v4.0:** 5 orchestrator patterns including Iterative (test-fix-retest) and Collaborative (multi-perspective review)
- **Persistent agent memory / scratchpad** — **DONE in v4.0:** ContextBridge provides per-session shared context between sequential phases
- **Agent cost estimation** — **PARTIALLY DONE in v4.0:** builtin_analyze_task estimates complexity and token counts pre-spawn
- **Agent attribution in chat** — messages attributed to originating agent with role label
- **Configurable agent roles** — user-defined roles beyond the 6 built-in ones
- **Model selection per role** — assign cheaper/faster models to Researchers, more capable models to Implementers
- **Cancel-all semantics** — option to discard completed results when user cancels "everything", not just running agents
- **Token budget enforcement** — ContextBridge.DEFAULT_BUDGET_FRACTION (0.3) exists but is not wired up. See §18.9
- **Conversation compaction** — sub-agent conversations grow unbounded up to 25 iterations. Mid-conversation summarization would reduce token waste. See §18.9

- **Remove debug logging** — `[TOKEN-SOURCE]`, `[RICA-PROXY][TOKEN-SOURCE]`, `[FileTracker]`, `[SpawnAgents]` `console.log` statements should be removed or gated behind a debug flag before release

17. v3.0 — Token Usage Tracking, API Usage Capture & Session Token UI

This section documents all features added after the v2.0 document (2026-04-15). The core goal: give users full visibility into token consumption across all modes (chat, agent, multi-agent) with real API usage data from Rica/Databricks rather than tiktoken estimation.

17.1 Problem Statement

Prior to v3.0, RICA had **zero token visibility** in the frontend. Token counts were calculated in `BaseLLM._logEnd()` using tiktoken (local estimation) and logged to SQLite + telemetry, but **never flowed back to the GUI**. The `PromptLog` interface lacked token fields entirely. Sub-agents consumed tokens with no tracking mechanism. Users had no way to know how many tokens a session consumed, whether counts were real or estimated, or how usage distributed across orchestrator vs. sub-agents.

Meanwhile, the Rica/Databricks API **already returns actual token usage** in every response — it was just being ignored.

17.2 Architecture: Two Data Paths

Token data flows through two independent pipelines that converge in a single frontend selector:

Path 1: Orchestrator / Chat / Agent tokens

Rica API response → `_lastApiUsage` (BaseLLM) → `PromptLog` return value
 → `ChatHistoryItem.promptLogs[]` → `Session.history` → disk
 → `selectSessionTokens` selector → `SessionTokenBadge` UI

Path 2: Sub-agent tokens

Rica API response → `_lastApiUsage` (BaseLLM) → `PromptLog` return value
 → `SubAgentRunner.callLLM()` captures `PromptLog`
 → Accumulated per-agent totals → `emitProgress()` with `tokenUsage`
 → `SubAgentProgress.tokenUsage` → `Session.subAgentProgress` → disk
 → `selectSessionTokens` selector → `SessionTokenBadge` UI + `MultiAgentPanel` cards

Both paths persist automatically — `PromptLog` tokens are saved in `ChatHistoryItem.promptLogs[]` within `Session.history`, and sub-agent tokens are saved in `Session.subAgentProgress`. The selector recomputes on-the-fly from persisted data when revisiting old sessions.

17.3 Real API Usage Capture (`_lastApiUsage`)

File: `core/llm/index.ts` (lines 186-194)

A new protected property on `BaseLLM` captures real usage from the most recent API response:

```
protected _lastApiUsage?: {
  promptTokens: number;
```

```

    completionTokens: number;
    reasoningTokens: number;
    totalTokens: number;
    cacheReadTokens: number;
    cacheCreationTokens: number;
  };

```

Lifecycle:

1. Cleared at the start of every `streamChat()` and `streamComplete()` call: `this._lastApiUsage = undefined;`
2. Populated at **4 capture points** (see 17.4)
3. Read at PromptLog return sites — if populated, real values used; if `undefined`, falls back to tiktoken estimation

17.4 Four Usage Capture Points

Capture Point 1: OpenAI Adapter — Non-Streaming (`core/llm/index.ts` ~line 968`)

```

if ((response as any).usage) {
  const u = (response as any).usage;
  this._lastApiUsage = {
    promptTokens: u.prompt_tokens ?? 0,
    completionTokens: u.completion_tokens ?? 0,
    reasoningTokens: u.reasoning_tokens ?? 0,
    totalTokens: u.total_tokens ?? (u.prompt_tokens ?? 0) + (u.completion_tokens ?? 0),
    cacheReadTokens: u.cache_read_input_tokens ?? 0,
    cacheCreationTokens: u.cache_creation_input_tokens ?? 0,
  };
}

```

Capture Point 2: OpenAI Adapter — Streaming (`core/llm/index.ts` ~line 994`)

Streaming requires `stream_options: { include_usage: true }` in the request body (added at line 989). The usage object appears on the **last chunk only**. Each chunk overwrites `_lastApiUsage`, so the final value has real completion counts.

Capture Point 3: RicaProxy — Non-Streaming (`core/llm/llms/stubs/RicaProxy.ts` ~line 367`)

Rica's non-streaming response includes `data.usage` with the same field names as OpenAI. Captured with debug logging:

```
[RICA-PROXY][TOKEN-SOURCE] Non-streaming API usage FOUND: {...}
```

An else branch logs when usage is missing:

```
[RICA-PROXY][TOKEN-SOURCE] Non-streaming API usage NOT found. Response keys: [...]
```

Capture Point 4: RicaProxy — Streaming (`core/llm/llms/stubs/RicaProxy.ts` ~line 396`)

Discovery: Rica streaming SSE chunks include `usage` on **every** chunk, but `completion_tokens` and `total_tokens` are `null` on intermediate chunks — only populated on the **last chunk**. Our code overwrites `_lastApiUsage` on every chunk (last one wins with

real values). Debug logging:

```
[RICA-PROXY][TOKEN-SOURCE] Streaming chunk usage FOUND: {...}
```

17.5 Rica API Usage Format

Rica/Databricks returns usage in OpenAI-compatible format with Anthropic-style cache fields:

```
{
  "usage": {
    "prompt_tokens": 786,
    "completion_tokens": 93,
    "total_tokens": 879,
    "reasoning_tokens": 116,
    "cache_read_input_tokens": 0,
    "cache_creation_input_tokens": 0
  }
}
```

Key findings during implementation:

- `reasoning_tokens` — present in response, represents thinking/reasoning tokens consumed by the model
- `cache_read_input_tokens` / `cache_creation_input_tokens` — Anthropic-style cache fields. Currently always `0` for the user's model/endpoint (caching not active), but the UI correctly hides cache display when values are 0
- Streaming: intermediate chunks have `completion_tokens: null`, `total_tokens: null`; only the last chunk has real values
- `prompt_tokens` is populated on all chunks (same value repeated)

17.6 PromptLog Type Extension

Files: `core/index.d.ts` (line 425), `core/config/types.ts` (line 389)

```
export interface PromptLog {
  modelTitle: string;
  completionOptions: CompletionOptions;
  prompt: string;
  completion: string;
  promptTokens?: number; // NEW v3.0
  completionTokens?: number; // NEW v3.0
  reasoningTokens?: number; // NEW v3.0
  totalTokens?: number; // NEW v3.0
  cacheReadTokens?: number; // NEW v3.0
  cacheCreationTokens?: number; // NEW v3.0
  /** true = actual API usage from provider; false/undefined = tiktoken estimation */
  isActualUsage?: boolean; // NEW v3.0
}
```

17.7 PromptLog Population (Prefer API, Fallback to Tiktoken)

File: `core/llm/index.ts` (lines 1076-1098)

Both `streamChat()` and `streamComplete()` return sites use the same pattern:

```
const hasApiUsage = !!this._lastApiUsage;
const promptTokens = this._lastApiUsage?.promptTokens ?? this.countTokens(prompt);
const completionTokens = this._lastApiUsage?.completionTokens ?? this.countTokens(completion);
const reasoningTokens = this._lastApiUsage?.reasoningTokens ?? 0;
const totalTokens = this._lastApiUsage?.totalTokens ?? (promptTokens + completionTokens);
const cacheReadTokens = this._lastApiUsage?.cacheReadTokens ?? 0;
const cacheCreationTokens = this._lastApiUsage?.cacheCreationTokens ?? 0;

return {
  modelTitle: this.title ?? completionOptions.model,
  prompt, completion, completionOptions,
  promptTokens, completionTokens, reasoningTokens, totalTokens,
  cacheReadTokens, cacheCreationTokens,
  isActualUsage: hasApiUsage,
};
```

The `isActualUsage` boolean allows the frontend to distinguish real API counts from tiktoken estimates.

17.8 SubAgentProgress Token Tracking

File: `core/index.d.ts` (line 481), `core/tools/multiagent/SubAgentRunner.ts`

Added `tokenUsage` to `SubAgentProgress` :

```
tokenUsage?: {
  promptTokens: number;
  completionTokens: number;
  reasoningTokens: number;
  totalTokens: number;
  cacheReadTokens: number;
  cacheCreationTokens: number;
  isActualUsage: boolean;
};
```

SubAgentRunner changes:

1. **Accumulators** (line 207-212): Six counters initialized at `run()` start:

```
let totalPromptTokens = 0;
let totalCompletionTokens = 0;
let totalReasoningTokens = 0;
let totalCacheReadTokens = 0;
let totalCacheCreationTokens = 0;
let hasActualUsage = false;
```

2. **callLLM() return type** (line 529): Extended to return all token fields:

```
Promise<{ message: ChatMessage; promptTokens: number; completionTokens: number;
  reasoningTokens: number; cacheReadTokens: number; cacheCreationTokens: number;
```



```
isActualUsage: boolean }>
```

3. **PromptLog capture** (line 551): The generator's return value (`result.value` when `result.done === true`) is the `PromptLog` . Previously ignored at line 526 — now captured and destructured.

4. **Accumulation** (lines 288-293): After each LLM call in the main loop:

```
totalPromptTokens += llmResult.promptTokens;
totalCompletionTokens += llmResult.completionTokens;
totalReasoningTokens += llmResult.reasoningTokens;
totalCacheReadTokens += llmResult.cacheReadTokens;
totalCacheCreationTokens += llmResult.cacheCreationTokens;
if (llmResult.isActualUsage) hasActualUsage = true;
```

5. **Progress emission** (lines 238-246): Every `emitProgress()` call includes the running `tokenUsage` totals, so the frontend sees live-updating counts as the agent works.

17.9 Session Token Selector

NEW file: `gui/src/redux/selectors/selectSessionTokens.ts` (137 lines)

A memoized Redux selector (`createSelector`) computing session-wide token totals from persisted data. Takes two inputs:

`state.session.history` and `state.session.subAgentProgress` .

Exports:

```
export interface LLMCallDetail {
  index: number;
  model: string;
  promptTokens: number;
  completionTokens: number;
  reasoningTokens: number;
  cacheReadTokens: number;
  cacheCreationTokens: number;
  totalTokens: number;
  isActualUsage: boolean;
}
```

Return shape:

```
{
  orchestratorTokens: {
    promptTokens, completionTokens, reasoningTokens,
    cacheReadTokens, cacheCreationTokens, total
  },
  agentTokens: { /* same shape */ },
  sessionTotal: number,      // orchestratorTotal + agentTotal
  totalReasoning: number,
  totalCacheRead: number,
  totalCacheCreation: number,
  usageSource: "api" | "estimated" | "mixed",
```

```
  llmCalls: LLMCallDetail[], // Per-call breakdown
  llmCallCount: number,
  agentCount: number,
}
```

How usageSource is determined:

- "api" — all calls have isActualUsage: true
- "estimated" — all calls used tiktoken (no API usage available)
- "mixed" — some calls have API usage, some don't (e.g., different providers or fallback scenarios)

17.10 Session Token Badge UI

File: gui/src/pages/gui/Chat.tsx (lines 80-234)

A clickable token counter visible in **all modes** (chat, agent, multi-agent), positioned at the top-right of the chat area.

Sub-components (local to file):

Component	Purpose
formatTokenCount(n)	Compact display: 879 / 1.2k / 45.3M
formatExact(n)	Locale-formatted for detail card: 1,234
SourceBadge	Colored pill: green "Real API Usage" / yellow "Mixed (API + Estimated)" / gray "Estimated (tiktoken)"
TokenRow	Reusable row for label/value pairs with optional color and indent. Returns null when value is 0 (auto-hides empty rows).
SessionTokenBadge	Main component with compact badge + expandable detail card

Compact badge: Plain clickable white text (not a button/panel), showing "{total} tokens {source}" . Source indicator: green "API" / yellow "~API" / gray "~est".

879 tokens API

Detail card (click to expand): Absolute-positioned dropdown with click-outside-to-close behavior:

Session Token Usage

[Real API Usage]

SUMMARY2 calls

Total879

Prompt (input)786← blue

Completion (output)93← green

Reasoning116← purple

Cache read (reused)--← sky (hidden when 0)

Cache write (new)--← sky (hidden when 0)

BREAKDOWN (if agents exist)

Orchestrator / Chat500

	Prompt	400	
	Completion	100	
	Sub-agents	379	
	Prompt	386	
	Completion	93	
	LLM CALLS	ranked by usage	
#1	Claude-4.5 Sonnet	411 / 84	API
#2	Gemini 3.1 Flash	375 / 9	API

LLM Calls section:

- Sorted by `totalTokens` descending (highest usage first)
- Numbered sequentially based on sort order (#1, #2, ...), not original call order
- Each row: `#{rank} {model} {prompt} / {completion} {API|est}`
- "ranked by usage" italic hint next to the section header
- Scrollable with `max-h-40` for long sessions

17.11 Per-Agent Token Display on MultiAgentPanel

File: `gui/src/pages/gui/MultiAgentPanel.tsx`

Agent Card Metrics Line (~line 628)

Compact token count appended after step count:

3 steps · 1.2k tok API

The `API / ~est` indicator uses green/gray coloring to match the session badge.

Agent Detail Overlay (~line 306)

Full token breakdown when expanded:

In: 786 Out: 93 R: 116 Cache: 0/0 Total: 879 tokens API

- `In:` = `promptTokens` (default zinc text)
- `Out:` = `completionTokens` (default zinc text)
- `R:` = `reasoningTokens` (purple, only shown when > 0)
- `Cache:` = `cacheRead/cacheCreation` (sky blue, only shown when either > 0, tooltip explains "read" vs "write")
- `Total:` = `totalTokens` (lighter zinc)
- `API / ~est` indicator (emerald / gray)

17.12 Modified Files Summary (v3.0)

File	Change	Version
core/index.d.ts	+7 fields on <code>PromptLog</code> (promptTokens, completionTokens, reasoningTokens, totalTokens, cacheReadTokens, cacheCreationTokens, isActualUsage); + <code>tokenUsage</code> object on <code>SubAgentProgress</code>	v3.0
core/config/types.ts	Mirror <code>PromptLog</code> token fields	v3.0
core/llm/index.ts	+ <code>_lastApiUsage</code> property; cleared in <code>streamChat/streamComplete</code> ; 2 OpenAI adapter capture points (non-streaming + streaming with <code>stream_options</code>); 2 <code>PromptLog</code> return sites with API-prefer-tiktoken-fallback pattern	v3.0
core/llm/llms/stubs/RicaProxy.ts	2 capture points (non-streaming + streaming); debug logging	v3.0
core/tools/multiagent/SubAgentRunner.ts	Token accumulators in <code>run()</code> ; extended <code>callLLM()</code> return type + <code>PromptLog</code> capture; <code>emitProgress()</code> includes <code>tokenUsage</code>	v3.0
gui/src/redux/selectors/selectSessionTokens.ts	NEW — memoized selector for session token totals, per-call breakdown, usage source detection	v3.0
gui/src/pages/gui/Chat.tsx	+ <code>SessionTokenBadge</code> component (compact badge + expandable detail card); + <code>formatTokenCount</code> , <code>formatExact</code> , <code>SourceBadge</code> , <code>TokenRow</code> helpers; imports <code>selectSessionTokens</code>	v3.0
gui/src/pages/gui/MultiAgentPanel.tsx	Per-agent token display on cards (metrics line) + detail overlay (full breakdown with reasoning, cache, source indicator)	v3.0

17.13 Persistence

Token counts persist automatically through existing mechanisms:

- **PromptLog tokens** — saved in `ChatHistoryItem.promptLogs[]` → `Session.history` → disk (via `HistoryManager.save()`)
- **SubAgentProgress tokens** — saved in `Session.subAgentProgress` → disk (added in v2.0)
- **Session totals** — computed on-the-fly by `selectSessionTokens` from persisted data. No separate storage needed.
- **Old sessions** — loading restores `history` + `subAgentProgress`, selector recomputes totals immediately. Sessions created before v3.0 will show 0 tokens (no `promptTokens` field in their `PromptLogs`).

17.14 Bugs Encountered & Resolutions (v3.0)

17.14.1 Rica Streaming Usage Format Discovery

Symptom: During initial implementation, it was assumed the streaming usage object would appear only on the last chunk (OpenAI behavior). Testing showed the `_lastApiUsage` was sometimes populated with `completion_tokens: 0`.

Root cause: Rica sends `usage` on **every** SSE chunk, but `completion_tokens` and `total_tokens` are `null` on intermediate chunks — only the last chunk has real values. The code was correctly overwriting `_lastApiUsage` on each chunk, but early during

debugging it was unclear why intermediate values showed `null`.

Resolution: No code change needed — the overwrite-on-every-chunk pattern naturally handles this (last chunk wins). Added detailed debug logging (`[RICA-PROXY][TOKEN-SOURCE]`) to confirm behavior. Documented the Rica streaming format for future reference.

Lesson: Don't assume all OpenAI-compatible APIs follow the exact same streaming behavior. Rica sends usage on every chunk; OpenAI sends it only on the last chunk (and only with `stream_options: { include_usage: true }`).

17.14.2 Token Badge Styled as Panel/Button

Symptom: User reported the session token badge looked like a button/panel, requested plain clickable text.

Root cause: Initial implementation used `<button>` with background color, hover effects, border-radius, and a chevron icon — making it look like a UI control.

Fix: Changed from `<button>` to `<div>` with `cursor-pointer`, `text-white/80`, removed background/hover/chevron. Now appears as plain text that happens to be clickable.

Lesson: Token/status indicators should be unobtrusive — plain text with `cursor-pointer` is more appropriate than button styling for always-visible metrics.

17.14.3 LLM Call Numbering After Sort

Symptom: After sorting LLM calls by usage (descending), the numbers showed original call order (#2, #1) instead of sorted rank (#1, #2).

Root cause: Used `call.index` (the original call sequence number) instead of the map iteration index after sorting.

Fix: Changed from `call.index` to `i + 1` where `i` is the `.map()` callback index:

```
{[...llmCalls].sort((a, b) => b.totalTokens - a.totalTokens).map((call, i) => (
  <span>#{i + 1}</span>
))}
```

17.14.4 Cache Tokens Always Zero

Symptom: Cache read and cache creation tokens consistently showed 0 in the UI despite being captured.

Root cause: Not a bug — the user's model/endpoint (Claude-4.5 Sonnet via Rica) does not have prompt caching active. The `cache_read_input_tokens` and `cache_creation_input_tokens` fields are present in Rica responses but always `0`.

Resolution: No code fix needed. The `TokenRow` component returns `null` when value is 0, so cache rows are automatically hidden. The UI will display them when caching is enabled for the endpoint.

17.15 Debug Logging Inventory (v3.0)

The following console.log statements were added for debugging and **remain in the codebase**. They should be removed or gated behind a debug flag before production release:

File	Log Prefix	Purpose
core/llm/index.ts:1085	[TOKEN-SOURCE] streamChat PromptLog:	Logs source (REAL API / TIKTOKEN EST) and all token counts at PromptLog return
core/llm/llms/stubs/RicaProxy.ts:369	[RICA-PROXY][TOKEN-SOURCE] Non-streaming API usage FOUND:	Confirms Rica non-streaming response has usage
core/llm/llms/stubs/RicaProxy.ts:379	[RICA-PROXY][TOKEN-SOURCE] Non-streaming API usage NOT found.	Warns when Rica non-streaming response lacks usage
core/llm/llms/stubs/RicaProxy.ts:396	[RICA-PROXY][TOKEN-SOURCE] Streaming chunk usage FOUND:	Logs every Rica streaming chunk that has usage (verbose — fires per chunk)

18. v4.0 — Batch Spawn, Orchestrator Patterns, Conflict Detection & Token Management

This section documents all features added after v3.0 (2026-04-15 → 2026-04-16). The core goals: (1) enable true batch agent spawning with a dedicated tool and UI, (2) provide the orchestrator with multiple execution patterns, (3) add infrastructure for agent coordination — ContextBridge, deduplication, retry, task analysis, and (4) implement pre-flight and post-flight file conflict detection.

18.1 Overview

v4.0 transforms RICA's multi-agent system from a basic "spawn one agent at a time" model into a production-grade orchestration platform with:

- **Batch spawning** — `builtin_spawn_agents` (plural) spawns multiple agents in a single tool call via `Promise.all()`
- **5 orchestrator patterns** — Auto, Sequential, Parallel, Iterative, Collaborative — each with tailored system messages
- **ContextBridge** — per-session shared context store for passing data between sequential agent phases
- **AgentManager** — centralized lifecycle management with task deduplication, scope extraction, and retry tracking
- **FaultHandler** — automatic retry with progressive backoff for transient errors
- **TaskDecomposer** — pre-spawn task analysis: complexity scoring, pattern suggestion, subtask decomposition, file overlap detection
- **FileTracker** — post-flight conflict detection for concurrent file edits
- **BatchAgentsDiv** — animated robot UI for batch spawn tool calls with proportional status coloring

18.2 Batch Agent Spawning (``builtin_spawn_agents``)

Problem: v1.0-v3.0 only had `builtin_spawn_agent` (singular). Spawning N agents required N separate tool calls, each processed sequentially by the streaming thunk. There was no true batch mechanism.

Solution: New `builtin_spawn_agents` (plural) tool that accepts an `agents` array and launches all agents simultaneously via `Promise.all()`.

Tool Definition

File: core/tools/definitions/spawnAgents.ts (70 lines)

```
Name: builtin_spawn_agents
Parameters:
  agents: Array of {
    task: string      - detailed task instructions
    description: string - short summary for UI display
    role?: string     - Researcher | Implementer | Tester | Reviewer | Analyst | Planner
    context?: string  - shared context from prior agents
  }
```

Implementation

File: core/tools/implementations/spawnAgents.ts (245 lines)

Flow:

- 1. Validate input (non-empty array, ≤ MAX_BATCH_SIZE=10)
- 2. Filter sub-agent tools (exclude SpawnAgent , SpawnAgents , AnalyzeTask from sub-agents)
- 3. For each agent spec:
 - Check for duplicates via AgentManager.checkDuplicate() (task hash matching)
 - If duplicate and completed → return cached result
 - If duplicate and running → return "already running" message
 - Else → register agent, push orchestrator context to ContextBridge
- 4. Promise.all(promises) — all non-duplicate agents run in parallel
- 5. Post-flight conflict detection via FileTracker.detectConflicts()
- 6. Build combined result with summary header + all agent results + conflict warnings

Registration in Tool System

- core/tools/builtIn.ts — SpawnAgents = "builtin_spawn_agents" enum value
- core/tools/callTool.ts — dispatch case for BuiltInToolNames.SpawnAgents
- core/tools/index.ts — spawnAgentsTool registered in baseToolDefinitions

18.3 Orchestrator Pattern System

Problem: v1.0-v3.0 had a single system message for the orchestrator that gave generic instructions. The orchestrator had no structured guidance on when to use parallel vs sequential execution, or how to handle iterative workflows.

Solution: 5 distinct orchestrator system messages, one per pattern, with shared building blocks and pattern-specific execution rules.

File: core/llm/multiAgentSystemMessage.ts (319 lines)

Shared Building Blocks

Block	Purpose
ORCHESTRATOR_PREAMBLE	Identity: "You are a multi-agent orchestrator"

Block	Purpose
DECISION_FRAMEWORK	When to reply directly vs spawn agents
AGENT_ROLES	Role definitions with parameter names
TASK_DESCRIPTIONS	Good vs bad task scoping examples
ADVANCED_CAPABILITIES	Task planning, context passing, naming, error handling
BASE_RULES	Core rules for all patterns

Pattern Descriptions

Pattern	Export	When Used	Key Rules
Auto	DEFAULT_MULTI_AGENT_SYSTEM_MESSAGE	Default — LLM picks best approach	5 execution modes described; "MINIMIZE AGENT COUNT" guidance; prefer single agent for ≤5 files
Sequential	SEQUENTIAL_SYSTEM_MESSAGE	Tasks with dependencies	One agent at a time; must continue the chain; pass context between phases
Parallel	PARALLEL_SYSTEM_MESSAGE	Independent subtasks	ONE call to builtin_spawn_agents with ALL agents; no overlap; split by area
Iterative	ITERATIVE_SYSTEM_MESSAGE	Test-fix-retest cycles	Phases: Investigate → Implement → Validate → retry (max 3 cycles)
Collaborative	COLLABORATIVE_SYSTEM_MESSAGE	Multi-perspective review	Multiple reviewers with DIFFERENT focus areas on SAME target; classify findings as CRITICAL/NON-CRITICAL

Agent Count Optimization

Added to Auto pattern to prevent unnecessary agent spawning:

```
MINIMIZE AGENT COUNT:
- Use the FEWEST agents that can accomplish the task well
- Do NOT split "gather info" and "analyze info" into separate agents
- Only split when subtasks are genuinely INDEPENDENT and LARGE enough
- For a folder with ≤5 files, ONE agent should handle the entire task
```

18.4 Advanced Agent Infrastructure

ContextBridge — Per-Session Shared Context

File: core/tools/multiagent/ContextBridge.ts (232 lines)

Per-session singleton that stores context entries for sharing between sequential agent phases. The orchestrator writes context; agents read it.


```
ContextBridge.forSession(sessionId)
  .pushContext(key, content, priority)    – orchestrator writes
  .getContextForAgent(agentId, budget?)  – agent reads (priority-sorted, budget-aware)
  .pushAgentResult(agentId, name, result) – system writes after agent completes
  .destroySession(sessionId)             – cleanup
```

- Token-aware: entries have `tokenCount` (estimated at ~4 chars/token)
- Auto-summarization: results >2000 tokens get extractive summaries (first 5 lines + last 3)
- Priority system: orchestrator context (priority 7) > agent results (priority 3)

Current limitation: `getContextForAgent()` is called without a `tokenBudget` parameter, so all context is included unbounded. See §18.9.

AgentManager — Lifecycle & Deduplication

File: `core/tools/multiagent/AgentManager.ts` (340 lines)

Global singleton managing agent records:

- **Registration:** `registerAgent()` creates `AgentRecord` with UUID, task hash, scope, display name
- **Deduplication:** `checkDuplicate()` matches `SHA256(role + ":" + normalizedTask)` against existing agents in same session
- **Scope extraction:** regex-based file/directory path extraction from task text
- **Display names:** auto-generated as `Role-N (scope)` (e.g., "Researcher-1 (frontend)")
- **Status tracking:** running → completed | errored | canceled

FaultHandler — Automatic Retry

File: `core/tools/multiagent/FaultHandler.ts` (217 lines)

Classifies errors and decides on retry strategy:

- **Retryable:** rate limits, timeouts, network errors → retry up to 3 times with progressive backoff (2s, 4s, 8s)
- **Non-retryable:** auth errors, permission failures, logical errors → escalate immediately
- **Escalation:** builds structured failure report with all attempt history and suggestions

runAgentWithRetry — Execution Wrapper

File: `core/tools/multiagent/runAgentWithRetry.ts` (116 lines)

Shared by both `spawnAgent` (singular) and `spawnAgents` (batch):

1. Creates `SubAgentRunner` with session ID and progress callback
2. Runs agent with full params (task, role, tools, model, display name, context bridge session)
3. On success → pushes result to ContextBridge for downstream agents
4. On error → FaultHandler classifies and decides retry/escalate
5. On cancel → respects cancellation, no retry

18.5 Task Analysis Tool (``builtin_analyze_task``)

File (definition): core/tools/definitions/analyzeTask.ts (39 lines) **File (implementation):** core/tools/implementations/analyzeTask.ts (28 lines) **File (logic):** core/tools/multiagent/TaskComposer.ts (559 lines)

Pre-spawn analysis tool that the orchestrator calls before spawning agents for complex tasks.

TaskComposer Analysis Pipeline

```
analyzeTask(task, ide, additionalPaths?)
├─ extractPaths(task)           - regex-based file/dir path extraction
├─ analyzeFiles(paths[], ide)   - read file sizes, line counts, token estimates
├─ estimateComplexity(task, files) - scoring: simple | moderate | complex
├─ suggestPattern(task)         - keyword-based: sequential | parallel | iterative | collaborative | auto
├─ decomposeTask(task, files)   - split into SubtaskPlans by numbered lists, bullets, conjunctions
├─ extractDependencies(subtasks) - heuristic: "based on" / "fix" implies dependency on prior subtask
├─ detectFileOverlaps(subtasks) - cross-check subtask scopes for shared files (v4.0)
└─ generateWarnings(files, task, overlaps) - large files, destructive ops, file conflicts (v4.0)
```

Complexity Scoring

Factor	Score
Task length > 100 words	+2
Task length > 50 words	+1
Files > 5	+2
Files > 2	+1
Total tokens > 50k	+2
Total tokens > 10k	+1
Individual file > 1000 lines	+1
Multiple actions (≥3 conjunctions)	+2
Result: simple (< 2), moderate (2-4), complex (≥ 5)	

Output Format

The orchestrator receives formatted markdown with:

- Complexity level and suggested pattern
- FILE CONFLICTS DETECTED section (if overlaps found) — see §18.6
- ACTION REQUIRED (if multiple files detected)
- File size table
- Suggested subtasks with roles, scopes, dependencies, token estimates
- Dependency graph

18.6 Pre-flight File Overlap Detection

Problem: The orchestrator could decompose a task into subtasks where two subtasks target the same file. Spawning parallel agents for overlapping files causes race conditions — the last write silently overwrites the first.

Solution: `TaskDecomposer.detectFileOverlaps()` cross-checks subtask scopes after decomposition. If two subtasks share a file path, a prominent warning is generated.

Implementation (in `TaskDecomposer.ts`):

```
private detectFileOverlaps(subtasks: SubtaskPlan[]): FileOverlap[] {
  // Build file → subtask[] map from each subtask's scope
  // Return entries where 2+ subtasks target the same normalized path
}
```

Warning format:

```
### FILE CONFLICTS DETECTED
- **manual-testing-sandbox/test.js** is targeted by subtasks T1, T2

**Action**: Merge conflicting subtasks into one agent, or switch to sequential pattern.
```

Orchestrator behavior (observed): Upon seeing this warning, the orchestrator merges conflicting subtasks into a single agent and runs non-conflicting subtasks in parallel. This eliminates ~90% of potential file conflicts before any agent runs.

18.7 Post-flight Conflict Detection (FileTracker)

Problem: Pre-flight detection catches planned overlaps, but LLM agents are unpredictable — an agent might edit files not mentioned in its original task scope.

Solution: `FileTracker` records every file edit with agent identity and content hashes. After `Promise.all()` completes, `detectConflicts()` finds files edited by multiple agents.

FileTracker

File: `core/tools/multiagent/FileTracker.ts` (81 lines)

Per-session singleton (same pattern as `ContextBridge`):

```
interface FileEdit {
  filepath: string;          // Normalized (forward slashes, lowercase)
  agentId: string;
  agentName: string;
  timestamp: number;
  contentHashBefore: string; // Hash of file when agent read it
  contentHashAfter: string;  // Hash of file after agent wrote it
}

interface FileConflict {
  filepath: string;
  edits: FileEdit[]; // All edits, sorted by timestamp
}

FileTracker.forSession(sessionId)
```

```
.recordEdit(edit)           – called by editFileForSubAgent after each write
.detectConflicts()          – returns files where 2+ different agents made edits
.destroySession(sessionId) – cleanup on killSession
```

Edit Recording Points

1. **EditExistingFile** — `editFileForSubAgent.ts` hashes content before/after write, records to FileTracker with agent ID, name, session ID, timestamp
2. **CreateNewFile** — `SubAgentRunner.ts` records creates to FileTracker after successful `callTool()` for CreateNewFile

Conflict Detection Flow

```
spawnAgents Promise.all() completes
├─ FileTracker.forSession(sessionId).detectConflicts()
│  ├─ Groups edits by normalized filepath
│  ├─ Filters to files with 2+ unique agent IDs
│  └─ Returns FileConflict[] sorted by filepath, edits sorted by timestamp

If conflicts found:
├─ Inserts warning ContextItem into results:
  "FILE CONFLICTS DETECTED: test.js edited by Implementer-1, Implementer-2
  Edit order: Implementer-1 (12:45:03) -> Implementer-2 (12:45:05)
  Action: Review files. Consider re-running with sequential pattern."
```

Session Cleanup

`core/core.ts` `killSession` handler calls `FileTracker.destroySession(sessionId)` alongside `ContextBridge.destroySession(sessionId)` and `SubAgentRegistry.cancelAllForSession(sessionId)`.

18.8 BatchAgentsDiv — Animated Robot UI

Problem: The `builtin_spawn_agents` tool call displayed as a generic spinner with raw JSON args in the chat UI.

Solution: Custom `BatchAgentsDiv` component with animated robot icons, floating popup card, and proportional status coloring.

File: `gui/src/pages/gui/ToolCallDiv/BatchAgentsDiv.tsx` (248 lines)

Layout

While running:

```
[robot1 robot2 robot3] 3 agents working...
```

- Up to 3 animated robot SVGs with staggered bounce animation (0s, 0.2s, 0.4s delay)
- `+N` badge if more than 3 agents
- Robots area is the only click target (not the full row)

While completed:

```
[robot1 robot2 robot3] 3 agents completed
```

- Static robots with proportional color coding

Floating Popup Card

Clicking robots opens a floating popup with:

- Header: "Parallel Agents" or "Cancelled by user"
- Count: `N/M done, K cancelled`
- Per-agent rows: chevron expand + status icon + role badge + description
- Expandable task detail per agent

Proportional Robot Coloring

When agents finish with mixed statuses (some completed, some cancelled, some errored), the 3 visible robots proportionally represent the distribution:

```
const errorSlots = erroredCount > 0 ? Math.ceil((erroredCount / agents.length) * shown) : 0;
const cancelSlots = canceledCount > 0 ? Math.ceil((canceledCount / agents.length) * shown) : 0;
// Priority: red (errored) → yellow (cancelled) → green (completed)
```

Examples:

- 7 agents, 1 cancelled → 1 yellow, 2 green robots
- 7 agents, 3 cancelled → 2 yellow, 1 green robots
- 7 agents, 7 cancelled → 3 yellow robots

Cancellation State

Derived from individual agent progress entries (not `toolCallState.status`, which shows "done" even when all agents were cancelled):

```
const hasCanceled = canceledCount > 0;
const allCanceled = agents.length > 0 && canceledCount === agents.length;
// Guard: agents.length > 0 prevents flash of "cancelled" while args are still parsing
```

CSS Animations

File: `gui/src/index.css`

```
@keyframes robot-work {
  0%, 100% { transform: translateY(0); }
  50% { transform: translateY(-3px); }
}
.animate-robot-1 { animation: robot-work 1.2s ease-in-out infinite; }
.animate-robot-2 { animation: robot-work 1.2s ease-in-out 0.2s infinite; }
.animate-robot-3 { animation: robot-work 1.2s ease-in-out 0.4s infinite; }
```

Registration

File: `gui/src/pages/gui/ToolCallDiv/FunctionSpecificToolCallDiv.tsx`

```
case BuiltInToolNames.SpawnAgents:
  return <BatchAgentsDiv toolCallState={toolCallState} />;
```

18.9 Token Management Analysis & Recommendations

A comprehensive analysis of token usage across the multi-agent system revealed three areas of inefficiency and corresponding recommendations.

Current Token Tracking (Working Well)

- SubAgentRunner tracks 6 token metrics per agent (prompt, completion, reasoning, total, cache read, cache creation)
- Tokens accumulated per LLM iteration and reported in SubAgentProgress
- `isActualUsage` flag distinguishes real API usage from tiktoken estimation
- `selectSessionTokens` memoized selector aggregates across orchestrator + all agents
- UI displays per-agent and session-level token counts

Inefficiency 1: Unbounded Context Inclusion (High Impact)

Problem: `ContextBridge.getContextForAgent()` is called in `SubAgentRunner.ts` without a `tokenBudget` parameter. The method defaults to returning ALL stored context entries.

```
// SubAgentRunner.ts:212 – no budget passed
const sharedContext = bridge.getContextForAgent(subAgentId);
```

`DEFAULT_BUDGET_FRACTION = 0.3` exists in `ContextBridge.ts` but is **never wired up**. In sequential patterns with many phases, each later agent receives the full cumulative output of all previous agents — growing unbounded.

Recommendation: Pass `model.contextLength * 0.3` as `tokenBudget`:

```
const sharedContext = bridge.getContextForAgent(subAgentId, model.contextLength * 0.3);
```

Estimated savings: 30-60% reduction in initial context for sequential agents beyond phase 2.

Inefficiency 2: No Conversation Compaction (Medium Impact)

Problem: SubAgentRunner appends messages indefinitely for up to 25 iterations. Tool results (often 500-1000 tokens each) accumulate in the conversation history. By iteration 15, every LLM call includes the full history of all prior iterations.

`compileChatMessages()` in `core/llm/index.ts` does prune old messages when the context window is exceeded, but this is a blunt instrument — it removes entire messages rather than summarizing them.

Recommendation: After every N iterations (e.g., 8), summarize older tool results into a compact "work done so far" message:

```
Iterations 1-8 summary:
- Read src/utils.ts (245 lines)
- Found 3 issues: missing null check L42, unused import L3, wrong return type L78
- Fixed null check at L42
```

Estimated savings: 10K-25K tokens per long-running agent (15+ iterations).

Inefficiency 3: All Tools Sent Every Request (Low Impact)

Problem: Every LLM call in SubAgentRunner includes all tool definitions (~10+ tokens per tool, repeated up to 25x). Tools the agent has never used and is unlikely to use still consume context.

Recommendation: After iteration 5, prune tool definitions to only include tools the agent has actually used + a small "discovery" set. Low priority — the per-request overhead is small.

Estimated savings: 5-10% per request (~100-500 tokens per iteration in a 25-iteration agent).

Token Metrics Summary

Parameter	Value	Location
MAX_TOOL_ITERATIONS	25	SubAgentRunner.ts:23
MAX_BATCH_SIZE	10 agents	spawnAgents.ts:18
SUMMARY_TOKEN_THRESHOLD	2000	ContextBridge.ts:20
DEFAULT_BUDGET_FRACTION	0.3 (unused)	ContextBridge.ts:23
DEFAULT_CONTEXT_LENGTH	8192	core/llm/constants.ts:2
DEFAULT_MAX_TOKENS	4096	core/llm/constants.ts:1
MIN_RESPONSE_TOKENS	1000	countTokens.ts:308
TOKEN_SAFETY_PROPORTION	0.02	countTokens.ts

18.10 New Files (v4.0)

File	Lines	Purpose
core/tools/multiagent/ContextBridge.ts	232	Per-session shared context store
core/tools/multiagent/AgentManager.ts	340	Agent lifecycle, deduplication, scope extraction
core/tools/multiagent/FaultHandler.ts	217	Error classification and retry logic
core/tools/multiagent/FileTracker.ts	81	Post-flight file conflict detection
core/tools/multiagent/TaskDecomposer.ts	559	Pre-spawn task analysis and decomposition
core/tools/multiagent/runAgentWithRetry.ts	116	Retry wrapper shared by singular/batch spawn

File	Lines	Purpose
core/tools/definitions/spawnAgents.ts	70	Tool definition for batch spawn
core/tools/definitions/analyzeTask.ts	39	Tool definition for task analysis
core/tools/implementations/spawnAgents.ts	245	Batch spawn implementation with conflict detection
core/tools/implementations/analyzeTask.ts	28	Task analysis implementation
core/llm/multiAgentSystemMessage.ts	319	5 orchestrator pattern system messages
gui/src/pages/gui/ToolCallDiv/BatchAgentsDiv.tsx	248	Animated robot UI for batch spawn
Total new	2,494	

18.11 Modified Files (v4.0)

File	Changes
core/tools/builtIn.ts	+ SpawnAgents , AnalyzeTask enum values
core/tools/callTool.ts	+dispatch cases for SpawnAgents, AnalyzeTask
core/tools/index.ts	+register spawnAgentsTool , analyzeTaskTool
core/tools/implementations/editFileForSubAgent.ts	+FileTracker import, simpleHash() , extended extras type with agentId/agentName/sessionId , content hashing before/after write, edit recording
core/tools/multiagent/SubAgentRunner.ts	+ FileTracker/resolveRelativePathInDir imports, pass agent identity to editFileForSubAgentImpl, track CreateNewFile writes to FileTracker
core/core.ts	+ FileTracker import, FileTracker.destroySession() in killSession handler
gui/src/pages/gui/ToolCallDiv/FunctionSpecificToolCallDiv.tsx	+BatchAgentsDiv import, SpawnAgents case in switch
gui/src/index.css	+robot-work keyframe animation, .animate-robot-1/2/3 classes

18.12 Updated Limitations & Future Work

Resolved in v4.0

Limitation	Resolution
No retry on failure	FaultHandler with 3 retries + progressive backoff
No agent coordination	ContextBridge for sequential context passing

Limitation	Resolution
No task analysis	builtin_analyze_task with TaskDecomposer
No conflict detection	Pre-flight (TaskDecomposer) + Post-flight (FileTracker)
Generic spawn UI	BatchAgentsDiv with animated robots and proportional status
Single execution pattern	5 patterns: Auto, Sequential, Parallel, Iterative, Collaborative
Over-spawning agents	"MINIMIZE AGENT COUNT" guidance + single-agent preference for small tasks

Remaining Limitations

- 1. **ContextBridge budget not enforced** — `DEFAULT_BUDGET_FRACTION` exists but `getContextForAgent()` called without budget. Unbounded context growth in sequential patterns.
- 2. **No conversation compaction** — sub-agent conversations grow up to 25 iterations without summarization.
- 3. **No model selection per role** — all sub-agents use the same LLM model as the orchestrator.
- 4. **Double path in resolveRelativePathInDir** — file URIs sometimes contain doubled directory segments (e.g., `manual-testing-sandbox/manual-testing-sandbox/test.js`). Not breaking (both agents get same doubled path, so conflict detection still works) but should be fixed.
- 5. **Debug logging remains** — `[FileTracker]` , `[SpawnAgents]` , `[TOKEN-SOURCE]` , `[RICA-PROXY]` `console.log` statements in production code.